The Concise TypeScript Book

Simone Poggiali

The Concise TypeScript Book

The Concise TypeScript Book предоставя изчерпателен и кратък преглед на възможностите на TypeScript. Книгата предлага ясни обяснения, които обхващат всички аспекти на най-новата версия на езика — от мощната му типова система до разширените функции.

Независимо дали сте начинаещ или опитен разработчик, тази книга е безценен ресурс за задълбочаване на познанията и уменията ви по TypeScript.

Тази книга е напълно безплатна и с отворен код.

Смятам, че висококачественото техническо образование трябва да бъде достъпно за всички. Затова предоставям книгата безплатно и я обновявам редовно с подобрения и нови примери.

Открийте **The Concise TypeScript Book Plus Edition**.



За читателите, които искат да надградят над изданието с отворен код, **The Concise TypeScript Book Plus Edition: React and Real-World Patterns for TypeScript 7** включва допълнително и ексклузивно съдържание, насочено към практическото приложение.

Разширеното издание включва:

- **Обновено за TypeScript 7** — обхваща най-новите функции и подобрения на езика в TypeScript 7.
- **TypeScript с React** — практически насоки за типизиране на компоненти, props, hooks, събития, children, refs и често използвани React шаблони.

- **TypeScript рецепти за реални проекти** — целенасочени примери за практически проблеми, с които разработчиците се сблъскват при създаването и поддръжката на TypeScript приложения.

С покупката на разширеното издание подкрепяте пряко и продължаващото развитие и поддръжка на безплатната книга с отворен код.

Разширеното издание е достъпно на английски и италиански език в Amazon по целия свят. Разгледайте разширеното издание и го купете от Amazon.

Подкрепете проекта

Ако безплатната книга ви е помогнала да отстраните грешка, да разберете сложна концепция или да напреднете в кариерата си, моля, обмислете да подкрепите работата ми, като платите колкото желаете, с препоръчителен принос от **$5**, или като ме почерпите с кафе.

Вашата подкрепа ми помага да поддържам съдържанието актуално и да го разширявам с нови примери, по-ясни обяснения и допълнителни практически насоки.





Преводи

Тази книга е преведена на няколко езика, включително:

английски

български

немски

френски

индонезийски

италиански

японски

корейски

полски

португалски (Бразилия)

шведски

турски

Vietnamese

Китайски

испански

тайландски

руски

арабски Чешки

Изтегляния и уебсайт

Можете също да изтеглите версията в Epub формат:

https://github.com/gibbok/typescript-book/tree/main/downloads

Онлайн версия е достъпна на:

https://gibbok.github.io/typescript-book

Съдържание

- The Concise TypeScript Book
  - Подкрепете проекта
  - Преводи
  - Изтегляния и уебсайт
  - Съдържание
  - Въведение
  - За автора
  - Въведение в TypeScript
    - Какво е TypeScript?
    - Защо TypeScript?
    - TypeScript и JavaScript
    - Генериране на код с TypeScript
    - Модерен JavaScript днес (Downleveling)
  - Първи стъпки с TypeScript
    - Инсталация
    - Конфигурация
    - Конфигурационен файл на TypeScript
      - target
      - lib
      - strict

        - module
        - moduleResolution
        - esModuleInterop
        - JSX option
        - skipLibCheck
        - files
        - include
        - exclude
    - importHelpers
    - Съвети за миграция към TypeScript
- Изследване на типовата система
    - Езиковата услуга на TypeScript
    - Структурна типизация
    - Основни правила за сравнение в TypeScript
    - Типовете като множества
    - Присвояване на тип: Декларации и проверки на типове
        - Декларация на тип
        - Проверка на тип (Type Assertion)
        - Ambient Declarations
    - Проверка на свойства и проверка за излишни свойства
    - Слаби типове
    - Строга проверка на обектни литерали (Freshness)
    - Извеждане на типове
    - По-сложни изводи
    - Разширяване на типовете
    - Const
        - Const Modifier on Type Parameters
        - Const assertion
    - Явна типова анотация

        - Стесняване на типове
            - Условия
            - Изключване на грешка или връщане от разклонение
            - Discriminated Union
            - Потребителски type guards
            - Switch-true narrowing
    - Примитивни типове
        - string
        - boolean
        - number
        - bigInt
        - Символ
        - null и undefined
        - Масив
        - any
    - Анотации на типовете
    - Опционални свойства
    - Readonly свойства
    - Сигнатури на индекси
    - Разширяване на типове
    - Literal Types
    - Literal Inference
    - strictNullChecks
    - Enums
        - Числови enums
        - Низови enums
        - Константни enums
        - Обратни съпоставки
        - Ambient enums
        - Изчисляеми и константни членове
    - Narrowing

        - Проверки за типа “typeof”
        - Свиване на тип чрез truthiness
        - Свиване на тип чрез равенство
        - Свиване на тип чрез оператора “in”
        - Свиване на тип чрез `instanceof`
    - Присвоявания
    - Анализ на потока на управление
    - Type Predicates
    - Discriminated Unions
    - The never Type
    - Проверка за изчерпателност
    - Обектни типове
    - Tuple тип (анонимен)
    - Именуван Tuple тип (с етикети)
    - Tuple с фиксирана дължина
    - Union тип
    - Intersection типове
    - Индексиране на тип
    - Тип от стойност
    - Тип от резултат на функция
    - Тип от модул
    - Mapped типове
    - Модификатори на Mapped типове
    - Conditional типове
    - Дистрибутивни Conditional типове
    - infer извеждане на тип в Conditional типове
    - Предефинирани Conditional типове
    - Template Union типове
    - Any тип
    - Unknown тип
    - Void тип
    - Never тип

- Interface и Type
  - Общ синтаксис
  - Основни типове
  - Обекти и Interfaces
  - Union и Intersection типове
- Вградени примитивни типове
- Често използвани вградени JS обекти
- Overloads
- Merging и Extension
- Разлики между Type и Interface
- Class
  - Общ синтаксис на Class
  - Constructor
  - Private и Protected конструктори
  - Модификатори за достъп
  - Get и Set
  - Auto-accessors в класове
  - this
  - Parameter Properties
  - Абстрактни класове
  - C generics
  - Decorators
    - Class Decorators
    - Property Decorator
    - Method Decorator
    - Декоратори за Getter и Setter
    - Metadata за декоратори
  - Наследяване
  - Статични членове
  - Инициализация на свойства
  - Method overloading
- Generics

        - Generic тип
        - Generic класове
        - Ограничения при generics
        - Контекстуално стесняване при generics
    - Изтрити структурни типове
    - Namespacing
    - Symbols
    - Triple-Slash директиви
    - Манипулация на типове
        - Създаване на типове от типове
        - Indexed Access Types
        - Utility типове
            - Awaited<T>
            - Partial<T>
            - Required<T>
            - Readonly<T>
            - Record<K, T>
            - Pick<T, K>
            - Omit<T, K>
            - Exclude<T, U>
            - Extract<T, U>
            - NonNullable<T>
            - Parameters<T>
            - ConstructorParameters<T>
            - ReturnType<T>
            - InstanceType<T>
            - ThisParameterType<T>
            - OmitThisParameter<T>
            - ThisType<T>
            - Uppercase<T>
            - Lowercase<T>
            - Capitalize<T>

        - Uncapitalize<T>
        - NoInfer<T>
- Други
    - Грешки и обработка на изключения
    - Mixin класове
    - Асинхронни функционалности
    - Итератори и генератори
    - TsDocs JSDoc Reference
    - @types
    - JSX
    - ES6 Modules
    - ES7 Exponentiation Operator
    - The for-await-of Statement
    - New target meta-property
    - Dynamic Import Expressions
    - “tsc –watch”
    - Non-null Assertion Operator
    - Декларации със стойности по подразбиране
    - Optional Chaining
    - Nullish coalescing operator
    - Template Literal Types
    - Function overloading
    - Recursive Types
    - Recursive Conditional Types
    - ECMAScript Module Support in Node
    - Assertion Functions
    - Variadic Tuple Types
    - Boxed types
    - Ковариантност и Контравариантност в TypeScript
        - Optional Variance Annotations for Type Parameters

- Template String Pattern Index Signatures
- The satisfies Operator
- Type-Only Imports и Export
- using declaration и Explicit Resource Management
  - Декларация await using
- Import Attributes
- Проверка на синтаксиса на регулярните изрази
- import defer

Въведение

Добре дошли в Книгата “The Concise TypeScript Book”! Този наръчник ви предоставя основни знания и практически умения за ефективно разработване с TypeScript. Открийте ключови концепции и техники за писане на чист и стабилен код. Независимо дали сте начинаещ или опитен разработчик, тази книга служи както като изчерпателен наръчник, така и като удобно справочно средство за използване на възможностите на TypeScript във вашите проекти.

Книгата обхваща TypeScript 7.0.

За автора

Simone Poggiali е опитен инженер с страст към писането на професионален код от 90-те години насам. През международната си кариера той е допринесъл за множество проекти за широк спектър от клиенти, от стартиращи компании до големи организации. Известни компании като HelloFresh, Siemens, O2, Leroy Merlin и Snowplow са се възползвали от неговия опит и отдаденост.

Можете да се свържете със Simone Poggiali на следните платформи:

- LinkedIn: https://www.linkedin.com/in/simone-poggiali
- GitHub: https://github.com/gibbok
- Bluesky: https://bsky.app/profile/gibbok.bsky.social
- Емейл: gibbok.coding gmail.com

Въведение в TypeScript

Какво е TypeScript?

TypeScript е строго типизиран език за програмиране, базиран на JavaScript. Той е създаден от Anders Hejlsberg през 2012 г. и в момента се разработва и поддържа от Microsoft като проект с отворен код.

TypeScript се компилира в JavaScript и може да се изпълнява във всяка JavaScript среда (например браузър или Node.js на сървър).

Поддържа множество програмни парадигми, като функционално, генерично, императивно и обектно-ориентирано програмиране, и е компилиран (транспилиран) език, който се преобразува в JavaScript преди изпълнение.

Защо TypeScript?

TypeScript е строго типизиран език, който помага за предотвратяване на често срещани грешки при програмирането и избягване на определени видове

грешки при изпълнението на програмата, преди тя да бъде изпълнена.

Строго типизираният език позволява на разработчика да задава различни ограничения и поведения на програмата чрез дефинициите на типовете данни. Това улеснява проверката на коректността на софтуера и предотвратяването на дефекти. Това е особено ценно при мащабни приложения.

Някои от предимствата на TypeScript:

- Статична типизация, по избор строго типизирана
- Извеждане на типове (Type Inference)
- Достъп до функционалности от ES6 и ES7
- Съвместимост между платформи и браузъри
- Поддръжка на инструменти с IntelliSense

TypeScript и JavaScript

TypeScript се пише във файлове с разширение `.ts` или `.tsx`, докато JavaScript файловете се пишат с разширение `.js` или `.jsx`.

Файловете с разширение `.tsx` или `.jsx` могат да съдържат JSX — разширение на синтаксиса на JavaScript, което се използва в React за разработка на потребителски интерфейси.

TypeScript е типизирано разширение на JavaScript (ECMAScript 2015) по отношение на синтаксиса. Всеки JavaScript код е валиден TypeScript код, но обратното не винаги е вярно.

Например, разгледайте функция в JavaScript файл с разширение `.js`, като следната:

```
const sum = (a, b) => a + b;
```

Функцията може да бъде преобразувана и използвана в TypeScript, като се промени разширението на файла на `.ts`. Ако обаче същата функция е анотирана с TypeScript типове, тя не може да бъде изпълнена в никоя JavaScript среда без компилация. Следният TypeScript код ще доведе до синтаксисна грешка, ако не бъде компилиран:

```
const sum = (a: number, b: number): number => a + b;
```

TypeScript е проектиран да открива възможни изключения, които могат да възникнат по време на изпълнение по време на компилация, като разработчикът дефинира намеренията си с типови анотации. Освен това TypeScript може да открива проблеми и ако не е предоставена типова анотация. Например, следният фрагмент от код не специфицира никакви типове TypeScript:

```
const items = [{ x: 1 }, { x: 2 }];
const result = items.filter(item => item.y);
```

В този случай TypeScript открива грешка и я докладва:

```
Property 'y' does not exist on type '{ x: number; }'.
```

Типовата система на TypeScript е силно повлияна от поведението на JavaScript по време на изпълнение. Например, операторът за събиране (+), който в JavaScript може да изпълнява както съединяване на низове, така и събиране на числа, е моделиран по същия начин в TypeScript:

```
const result = '1' + 1; // Резултатът е от тип string
```

Екипът, стоящ зад TypeScript, е взел съзнателно решение да маркира необичайното използване на JavaScript като грешки. Например, разгледайте следния валиден JavaScript код:

```
const result = 1 + true; // В JavaScript резултатът е равен на 2
```

Въпреки това, TypeScript хвърля грешка:

```
Operator '+' cannot be applied to types 'number' and 'boolean'.
```

Тази грешка възниква, защото TypeScript строго налага съвместимост на типовете и в този случай идентифицира невалидна операция между число и булева стойност.

Генериране на код с TypeScript

Компилаторът на TypeScript има две основни отговорности: проверка за грешки в типовете и компилиране към JavaScript. Тези два процеса са независими един от друг. Типовете не влияят върху изпълнението на кода в JavaScript среда за изпълнение, тъй като са напълно премахнати по време на компилацията. TypeScript все пак може да генерира JavaScript дори при наличие на грешки в типовете. Ето пример за TypeScript код с грешка в типа:

```
const add = (a: number, b: number): number => a + b;
const result = add('x', 'y'); // Argument of type 'string' is not assignable to parameter of type 'number'.
```

Въпреки това, TypeScript все още може да генерира изпълним JavaScript код:

```
'use strict';
const add = (a, b) => a + b;
const result = add('x', 'y'); // xy
```

Не е възможно да се проверяват типовете на TypeScript по време на изпълнение. Например:

```
interface Animal {
    name: string;
}
interface Dog extends Animal {
    bark: () => void;
}
interface Cat extends Animal {
    meow: () => void;
}
const makeNoise = (animal: Animal) => {
    if (animal instanceof Dog) {
        // 'Dog' е само тип, но тук се използва като стойност.
        // ...
    }
};
```

Тъй като типовете се изтриват след компилация, няма начин да се изпълни този код в JavaScript. За да разпознаем типовете по време на изпълнение, трябва да използваме друг механизъм. TypeScript предоставя няколко опции, като една от често използваните е “tagged union”. Например:

```
interface Dog {
    kind: 'dog'; // Tagged union
    bark: () => void;
}
```

```
interface Cat {
    kind: 'cat'; // Tagged union
    meow: () => void;
}
type Animal = Dog | Cat;

const makeNoise = (animal: Animal) => {
    if (animal.kind === 'dog') {
        animal.bark();
    } else {
        animal.meow();
    }
};

const dog: Dog = {
    kind: 'dog',
    bark: () => console.log('bark'),
};
makeNoise(dog);
```

Свойството “kind” е стойност, която може да се използва по време на изпълнение, за да се разграничат обектите в JavaScript.

Възможно е също така стойността по време на изпълнение да има тип, различен от този, деклариран в декларацията за типа. Например, ако разработчикът е интерпретирал погрешно типа на API и го е анотирал неправилно.

TypeScript е надмножество (superset) на JavaScript, така че ключовата дума “class” може да се използва както като тип, така и като стойност по време на изпълнение.

```
class Animal {
    constructor(public name: string) {}
}
class Dog extends Animal {
```

```
    constructor(
        public name: string,
        public bark: () => void
    ) {
        super(name);
    }
}
class Cat extends Animal {
    constructor(
        public name: string,
        public meow: () => void
    ) {
        super(name);
    }
}
type Mammal = Dog | Cat;

const makeNoise = (mammal: Mammal) => {
    if (mammal instanceof Dog) {
        mammal.bark();
    } else {
        mammal.meow();
    }
};

const dog = new Dog('Fido', () => console.log('bark'));
makeNoise(dog);
```

В JavaScript “class” има свойство “prototype”, а операторът “instanceof” може да се използва, за да се провери дали свойството prototype на даден конструктор се среща някъде в прототипната верига на обект.

TypeScript няма влияние върху производителността по време на изпълнение, тъй като всички типове се премахват. Въпреки това TypeScript добавя известно натоварване по време на компилация (build time).

Модерен JavaScript днес (Downleveling)

TypeScript може да компилира код към всяка публикувана версия на JavaScript, започвайки от ECMAScript 3 (1999). Това означава, че TypeScript може да транспилира код, използващ най-новите JavaScript функционалности, към по-стари версии - процес, известен като Downleveling. Това позволява използването на модерен JavaScript, като същевременно се поддържа максимална съвместимост с по-стари среди за изпълнение.

Важно е да се отбележи, че по време на транспилацията към по-стара версия на JavaScript, TypeScript може да генерира код, който да доведе до забавяне на производителността в сравнение с нативните имплементации.

Ето някои от модерните JavaScript функционалности, които могат да се използват в TypeScript:

- ECMAScript модули вместо AMD-стил “define” callback-и или CommonJS “require” изрази.
- Класове вместо прототипи.
- Деклариране на променливи с “let” или “const” вместо “var”.
- “for-of” цикъл или “.forEach” вместо традиционния “for” цикъл.
- Arrow функции вместо функционални изрази.
- Деструктуриране присвояване.
- Съкратени имена на свойства/методи и изчислени имена на свойства.
- Параметри по подразбиране на функции.

Използвайки тези модерни функции на JavaScript, разработчиците могат да пишат по-изразителен и кратък код в TypeScript.

Първи стъпки с TypeScript

Инсталация

Visual Studio Code предоставя отлична поддръжка за езика TypeScript, но не включва компилатора на TypeScript. За да инсталирате компилатора на TypeScript, можете да използвате пакетен мениджър като npm или yarn:

```
npm install typescript --save-dev
```

or

```
yarn add typescript --dev
```

Уверете се, че сте записвали генерирания файл за заключване (lockfile), за да гарантирате, че всеки член на екипа използва една и съща версия на TypeScript.

За да стартирате компилатора TypeScript, можете да използвате следните команди

```
npx tsc
```

or

```
yarn tsc
```

Препоръчително е да инсталирате TypeScript за всеки проект, а не глобално, тъй като това осигурява по-предвидим процес на изграждане. Въпреки това, за

еднократни случаи, можете да използвате следната команда:

```
npx tsc
```

или инсталирате глобално:

```
npm install -g typescript
```

Ако използвате Microsoft Visual Studio, можете да получите TypeScript като пакет в NuGet за вашите MSBuild проекти. В конзолата NuGet Package Manager Console изпълнете следната команда:

```
Install-Package Microsoft.TypeScript.MSBuild
```

По време на инсталирането на TypeScript се инсталират два изпълними файла: “tsc” като компилатор на TypeScript и “tsserver” като самостоятелен сървър на TypeScript. Самостоятелният сървър съдържа компилатора и езиковите услуги, които могат да се използват от редакторите и IDE-тата, за да осигурят интелигентно попълване на кода.

Освен това има няколко транспайлъра, съвместими с TypeScript, като Babel (чрез плъгин) или swc. Тези транспайлъри могат да се използват за преобразуване на TypeScript код в други целеви езици или версии.

TypeScript 7.0 беше пренаписан в Go като нативна имплементация на компилатора и езиковата услуга. Той използва многонишковост със споделена памет и други оптимизации, за да направи пълните изграждания и

функционалностите в редактора по-бързи, като намалява времето за обратна връзка по време на разработка.

Някои функции за производителност в TypeScript 7.0 могат да се настройват. Проверката на типовете може да се изпълнява в паралелни worker-и с `--checkers`; повече worker-и могат да ускорят големи проекти, но използват повече памет. Преработеният режим `--watch` подобрява наблюдението на файлове между различни платформи. TypeScript 7.0 все още не включва compiler API (към юли 2026 г.), така че инструменти, които все още се нуждаят от TypeScript 6.0 API, могат да работят паралелно с TypeScript 7.0 чрез `@typescript/typescript6` или npm aliases.

Конфигурация

TypeScript може да бъде конфигуриран чрез CLI опциите на tsc или чрез използването на специален конфигурационен файл, наречен tsconfig.json, който се поставя в основната директория (root) на проекта.

За да генерирате файл tsconfig.json, предварително попълнен с препоръчителни настройки, можете да използвате следната команда:

```
tsc --init
```

Когато изпълнявате командата `tsc` локално, TypeScript ще компилира кода, използвайки конфигурацията, зададена в най-близкия файл tsconfig.json.

Ето няколко примера за CLI команди, които се изпълняват с настройките по подразбиране:

```
tsc main.ts // Compile a specific file (main.ts) to JavaScript
tsc src/*.ts // Compile any .ts files under the 'src' folder to JavaScript
tsc app.ts util.ts --outfile index.js // Compile two TypeScript files (app.ts and util.ts) into a single JavaScript file (index.js)
```

Конфигурационен файл на TypeScript

Файлът tsconfig.json се използва за конфигуриране на компилатора TypeScript (tsc). Обикновено той се добавя в основната директория на проекта, заедно с файла `package.json`.

Забележки:

- tsconfig.json има възможност за коментари, дори ако е в json формат.
- Препоръчително е да използвате този конфигурационен файл вместо опциите на командния ред.

На следния линк можете да намерите пълната документация и нейната схема:

https://www.typescriptlang.org/tsconfig

https://www.typescriptlang.org/tsconfig/

По-долу е представен списък с често използвани и полезни конфигурации:

target

Свойството “target” се използва, за да се определи в коя версия на ECMAScript трябва да се генерира/компилира вашият TypeScript код. За съвременните браузъри ES6 е добър избор. Забележка: Поддръжката на ES5 беше обявена за остаряла в TypeScript 6.0 и вече не се поддържа в TypeScript 7.0.

lib

Свойството “lib” се използва, за да се определи кои библиотечни файлове да се включат по време на компилацията. TypeScript автоматично включва API за функции, определени в свойството “target”, но е възможно да се пропуснат или да се изберат конкретни библиотеки за специфични нужди. Например, ако работите по сървър проект, можете да изключите библиотеката “DOM”, която е полезна само в среда с браузър.

strict

Опцията “strict” подобрява типова безопасността, като позволява по-строги проверки. Тя е активирана по подразбиране, считано от TypeScript 6.0; в противен случай трябва изрично да я зададете на true във вашия файл tsconfig.json. Активирането на “strict” позволява на TypeScript да:

- Генерира код, използвайки “use strict” за всеки изходен файл.
- Взема предвид “null” и “undefined” в процеса на проверка на типовете.

- Деактивира използването на типа “any”, когато няма типови анотации.
- Показва грешка при използването на израза “this”, който иначе би означавал типа “any”.

module

Свойството “module” задава модулната система, която се поддържа от компилираната програма. По време на изпълнение се използва модулен зареждач (module loader), който намира и изпълнява зависимостите според зададената модулна система.

Най-често използваните модулни зареждачи в JavaScript са Node.js CommonJS за сървърни приложения и RequireJS за AMD модули в уеб приложения, базирани на браузър. TypeScript може да генерира код за различни модулни системи, включително UMD, System, ESNext, ES2015/ES6 и ES2020. Модулната система трябва да се избира в зависимост от целевата среда и наличния механизъм за зареждане на модули в тази среда.

Забележка: Поддръжката на по-старите модулни системи (AMD, UMD, SystemJS) беше обявена за остаряла в TypeScript 6.0 и вече не се поддържа в TypeScript 7.0.

moduleResolution

Свойството “moduleResolution” определя стратегията за разрешаване на модули. Използвайте “nodenext” или “bundler” за модерен TypeScript код. Стратегията “classic” се използва само за стари версии на TypeScript (преди 1.6).

esModuleInterop

Свойството “esModuleInterop” позволява импортиране на подразбиращи се експорти от CommonJS модули, които не са експортирали с помощта на свойството “default”. Това свойство предоставя shim, за да се осигури съвместимост в генерирания JavaScript. След активиране на тази опция можем да използваме `import MyLibrary from "my-library"` вместо `import * as MyLibrary from "my-library"`.

Функциите “esModuleInterop” първоначално се активираха по избор, за да се избегнат промени, нарушаващи съвместимостта, но отдавна са препоръчителните настройки по подразбиране. Деактивирането им може да доведе до незабележими проблеми при изпълнението при използване на CommonJS с ESM. Забележка: Началото с TypeScript 6.0 това по-безопасно поведение при взаимодействието е винаги активирано.

В TypeScript 6.0 някои по-стари конфигурационни опции и синтактични форми бяха обявени за остарели или преминаваха през старо поведение. В TypeScript 7.0 те са директни грешки или поведение без ефект.

Остарелите възможности, които са се превърнали в директни грешки с поведение без ефект, са:

- `target: es5`
- `downlevelIteration`
- `moduleResolution: node/node10`
- `module: amd/umd/systemjs/none`
- `baseUrl`
- `moduleResolution: classic`

- деактивиране на `esModuleInterop` или `allowSyntheticDefaultImports`
- деактивиране на `alwaysStrict`
- ключовата дума `module` в декларации на namespace
- `asserts` при imports
- `/// <reference no-default-lib />` под `skipDefaultLibCheck`
- файлови пътища в CLI с локален `tsconfig.json`, освен ако не се използва `--ignoreConfig`

JSX option

Свойството “jsx” се прилага само за .tsx файлове, използвани в ReactJS, и контролира как JSX конструкциите се компилират в JavaScript. Често използвана опция е “preserve”, която ще компилира в .jsx файл, като запази JSX непроменен, за да може да бъде предаден на различни инструменти като Babel за по-нататъшни трансформации.

skipLibCheck

Свойството “skipLibCheck” ще попречи на TypeScript да проверява типовете на всички импортирани външни пакети. Това ще намали времето за компилация на проекта. TypeScript обаче все още ще проверява вашия код спрямо дефинициите на типовете, предоставени от тези пакети.

files

Свойството “files” указва на компилатора списък с файлове, които винаги трябва да бъдат включени в програмата.

include

Свойството “include” указва на компилатора списък с файлове, които искаме да включим. Това свойство позволява използването на glob-подобни шаблони, като “*” *за всяка поддиректория,* ”” за всяко име на файл и “?” за опционални символи.

exclude

Свойството “exclude” указва на компилатора списък с файлове, които не трябва да бъдат включени в компилацията. Това може да включва файлове като “node_modules” или тестови файлове. Забележка: tsconfig.json позволява коментари.

importHelpers

TypeScript използва помощен код при генериране на код за определени усъвършенствани или down-level JavaScript функционалности. По подразбиране тези помощни функции се дублират във файловете, които ги използват. Опцията `importHelpers` вместо това импортира тези помощни функции от модула `tslib`, което прави изходния JavaScript по-ефективен.

Съвети за миграция към TypeScript

При големи проекти се препоръчва постепенна миграция, при който кодът на TypeScript и JavaScript първоначално ще съществуват едновременно. Само малки проекти могат да бъдат мигрирани към TypeScript наведнъж.

Първата стъпка в този преход е въвеждането на TypeScript в процеса на изграждане. Това може да се направи чрез използване на опцията на компилатора “allowJs”, която позволява на файловете .ts и .tsx да са заедно със съществуващите JavaScript файлове. Тъй като TypeScript ще използва типа “any” за променлива, когато не може да изведе типа от JavaScript файловете, се препоръчва да деактивирате “noImplicitAny” в опциите на компилатора в началото на миграцията.

Втората стъпка е да се уверите, че вашите JavaScript тестове работят заедно с TypeScript файловете, така че да можете да изпълнявате тестове, докато конвертирате всеки модул. Ако използвате Jest, обмислете използването на `ts-jest`, което ви позволява да тествате TypeScript проекти с Jest.

Третата стъпка е да включите декларации за типове за външни библиотеки във вашия проект. Тези декларации могат да бъдат намерени или вградени в самите библиотеки, или в DefinitelyTyped. Можете да ги търсите чрез https://www.typescriptlang.org/dt/search и да ги инсталирате, като използвате:

```
npm install --save-dev @types/package-name
```

or

```
yarn add --dev @types/package-name.
```

Четвъртата стъпка е да мигрирате модул по модул, като използвате подход “отдолу нагоре” и следвате графика на зависимостите, започвайки от крайните възли. Идеята е да започнете с преобразуването на модулите, които не

зависят от други модули. За да визуализирате графиките на зависимостите, можете да използвате инструмента “madge”.

Подходящи модули за тези първоначални преобразувания са помощните функции и кодът, свързан с външни API-та или спецификации. Възможно е автоматично да генерирате дефиниции на типове в TypeScript от Swagger договори, GraphQL или JSON схеми, които да бъдат включени във вашия проект.

Когато няма налични спецификации или официални схеми, можете да генерирате типове от необработени данни, като например JSON, върнат от сървър. Препоръчително е обаче да генерирате типове от спецификации, а не от данни, за да избегнете пропускане на крайни случаи.

По време на миграцията се въздържайте от рефакториране на кода и се фокусирайте единствено върху добавянето на типове към вашите модули.

Петата стъпка е да активирате “noImplicitAny”, което ще наложи всички типове да бъдат известни и дефинирани, осигурявайки по-добра работа с TypeScript за вашия проект.

По време на миграцията можете да използвате директивата `@ts-check`, която активира проверката на типовете в TypeScript в JavaScript файл. Тази директива осигурява по-свободна форма на проверката на типовете и може първоначално да се използва за идентифициране на проблеми в JavaScript файлове. Когато `@ts-check` е включена

във файл, TypeScript ще се опита да изведе дефиниции, използвайки коментари в стил JSDoc. Въпреки това, помислете за използването на JSDoc анотации само в много ранен етап от миграцията.

Също така е добре да запазите стойността по подразбиране на `noEmitOnError` в tsconfig.json като false. Това ще позволи генерирането на JavaScript код дори ако бъдат отчетени грешки.

Изследване на типовата система

Езиковата услуга на TypeScript

Езиковата услуга на TypeScript, известна още като tsserver, предлага различни функции, като отчитане на грешки, диагностика, компилиране при запазване, преименуване, преминаване към дефиниция, списъци за автодопълване, помощ за сигнатури и други. Тя се използва предимно от интегрираните среди за разработка (IDE), за да осигурява поддръжка на IntelliSense. Тя се интегрира безпроблемно с Visual Studio Code и се използва от инструменти като Conquer of Completion (Coc).

Разработчиците могат да използват специален API и да създават свои собствени плъгини за езиковата услуга, за да подобрят опита при редактиране на TypeScript. Това може да бъде особено полезно при имплементация на специални функции за проверка на кода или за активиране на автодопълване за персонализиран език за шаблони.

Пример за реален потребителски плъгин е “typescript-styled-plugin”, който предоставя докладване на синтактични грешки и IntelliSense поддръжка за CSS свойства в styled components.

За повече информация и ръководства за бързо начало можете да се обърнете към официалната документация на TypeScript Wiki в GitHub: https://github.com/microsoft/TypeScript/wiki/

Структурна типизация

TypeScript се базира на структурна типова система. Това означава, че съвместимостта и еквивалентността на типовете се определят от реалната структура или дефиниция на типа, а не от името му или мястото на декларация, както при номинативните типови системи като C# или C.

Структурната типова система на TypeScript е проектирана въз основа на това как работи динамичната duck typing система на JavaScript по време на изпълнение.

Следният пример е валиден TypeScript код. Както може да се види, “X” и “Y” имат един и същи член “a”, въпреки че имат различни имена на декларациите. Типовете се определят от техните структури и в този случай, тъй като структурите са еднакви, те са съвместими и валидни.

```
type X = {
    a: string;
};
type Y = {
    a: string;
```

```
};
const x: X = { a: 'a' };
const y: Y = x; // Валидно
```

Основни правила за сравнение в TypeScript

Процесът на сравнение в TypeScript е рекурсивен и се прилага върху типове, вложени на всяко ниво.

Типът “X” е съвместим с “Y”, ако “Y” има поне същите елементи като “X”.

```
type X = {
    a: string;
};
const y = { a: 'A', b: 'B' }; // Вярно, тъй като има поне същите членове като X
const r: X = y;
```

Параметрите на функциите се сравняват по типове, а не по имена:

```
type X = (a: number) => void;
type Y = (a: number) => void;
let x: X = (j: number) => undefined;
let y: Y = (k: number) => undefined;
y = x; // Валидно
x = y; // Валидно
```

Типовете на връщаните стойности от функции трябва да са еднакви:

```
type X = (a: number) => undefined;
type Y = (a: number) => number;
let x: X = (a: number) => undefined;
let y: Y = (a: number) => 1;
```

```
y = x; // Невалидно
x = y; // Невалидно
```

Типът на връщаната стойност на изходната функция трябва да бъде подтип на типа на връщаната стойност на целевата функция:

```
let x = () => ({ a: 'A' });
let y = () => ({ a: 'A', b: 'B' });
x = y; // Валидно
y = x; // Липсва невалиден елемент b
```

Допуска се изключването на параметри на функции, тъй като това е обичайна практика в JavaScript, например при използването на “Array.prototype.map()”:

```
[1, 2, 3].map((element, _index, _array) => element + 'x');
```

Следователно следните декларации на типове са напълно валидни:

```
type X = (a: number) => undefined;
type Y = (a: number, b: number) => undefined;
let x: X = (a: number) => undefined;
let y: Y = (a: number) => undefined; // Липсва невалиден параметър b
y = x; // Валидно
```

Всички допълнителни опционални параметри на изходния тип са валидни:

```
type X = (a: number, b?: number, c?: number) => undefined;
type Y = (a: number) => undefined;
let x: X = a => undefined;
let y: Y = a => undefined;
y = x; // Валидно
x = y; //Валидно
```

Всички опционални параметри на целевия тип без съответстващи параметри в изходния тип са валидни и не са грешка:

```
type X = (a: number) => undefined;
type Y = (a: number, b?: number) => undefined;
let x: X = a => undefined;
let y: Y = a => undefined;
y = x; // Валидно
x = y; // Валидно
```

Параметърът “rest” се третира като безкрайна поредица от опционални параметри:

```
type X = (a: number, ...rest: number[]) => undefined;
let x: X = a => undefined; //валидно
```

Функциите с overloads са валидни, ако сигнатурата на overload е съвместима със сигнатурата на неговата реализация:

```
function x(a: string): void;
function x(a: string, b: number): void;
function x(a: string, b?: number): void {
    console.log(a, b);
}
x('a'); // Валидно
x('a', 1); // Валидно

function y(a: string): void; // Невалидно, несъвместимо със сигнатурата на реализацията
function y(a: string, b: number): void;
function y(a: string, b: number): void {
    console.log(a, b);
}
y('a');
y('a', 1);
```

Сравнението на параметрите на функциите е успешно, ако изходните и целевите параметри могат да бъдат присвоени към супертипове или подтипове (bivariance).

```
// Supertype
class X {
    a: string;
    constructor(value: string) {
        this.a = value;
    }
}
// Subtype
class Y extends X {}
// Subtype
class Z extends X {}

type GetA = (x: X) => string;
const getA: GetA = x => x.a;

// Bivariance does accept supertypes
console.log(getA(new X('x'))); // Валидно
console.log(getA(new Y('Y'))); // Валидно
console.log(getA(new Z('z'))); // Валидно
```

Enum-овете могат да се сравняват и са валидни с числа и обратно, но сравняването на стойности от различни Enum типове е невалидно.

```
enum X {
    A,
    B,
}
enum Y {
    A,
    B,
    C,
}
```

```
const xa: number = X.A; // Валидно
const ya: Y = 0; // Валидно
X.A === Y.A; // Невалидно
```

Инстанциите на даден клас се подлагат на проверка за съвместимост по отношение на частните и защитените им елементи:

```
class X {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}

class Y {
    private a: string;
    constructor(value: string) {
        this.a = value;
    }
}

let x: X = new Y('y'); // Невалидно
```

При сравнителната проверка не се взема предвид разликата в йерархията на наследяване, например:

```
class X {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}
class Y extends X {
    public a: string;
    constructor(value: string) {
        super(value);
        this.a = value;
```

```
    }
}
class Z {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}
let x: X = new X('x');
let y: Y = new Y('y');
let z: Z = new Z('z');
x === y; // Валидно
x === z; // Валидно дори ако z е от друга йерархия на наследяване
```

Generics се сравняват чрез техните структури въз основа на резултатния тип след прилагане на generic параметъра; сравнява се само крайният резултат като не-generic тип.

```
interface X<T> {
    a: T;
}
let x: X<number> = { a: 1 };
let y: X<string> = { a: 'a' };
x === y; // Невалидно тъй като аргументът за типа се използва в крайната структура

interface X<T> {}
const x: X<number> = 1;
const y: X<string> = 'a';
x === y; // Валидно тъй като аргументът за типа не се използва в крайната структура
```

Когато generics нямат зададен аргумент за тип, всички незададени аргументи се третират като типове “any”:

```
type X = <T>(x: T) => T;
type Y = <K>(y: K) => K;
let x: X = x => x;
```

```
let y: Y = y => y;
x = y; // Валидно
```

Не забравяйте:

```
let a: number = 1;
let b: number = 2;
a = b; // Валидно, всичко може да се присвои на себе си

let c: any;
c = 1; // Валидно, всички типове могат да се присвоят на any

let d: unknown;
d = 1; // Валидно, всички типове могат да се присвоят на unknown

let e: unknown;
let e1: unknown = e; // Валидно, unknown може да бъде присвоен само на себе си и на any
let e2: any = e; // Валидно
let e3: number = e; // Невалидно

let f: never;
f = 1; // Невалидно, нищо не може да се присвои на never

let g: void;
let g1: any;
g = 1; // Невалидно, променливата void не може да бъде присвоявана към или от нищо, освен от самата себе си
g = g1; // Валидно
```

Имайте предвид, че когато е активирана опцията “strictNullChecks”, “null” и “undefined” се третират по подобен начин като “void”; а в противен случай те се третират като “never”.

Типовете като множества

В TypeScript типът е множеството от възможни стойности. Това множество се нарича още домейн на типа. Всяка стойност на типа може да се разглежда като елемент в множеството. Типът определя ограниченията, които всеки елемент в множеството трябва да удовлетворява, за да се счита за член на това множество. Основната задача на TypeScript е да проверява и потвърждава дали едно множество е подмножество на друго.

TypeScript поддържа различни типове множества:

Термин от теорията на множествата	**TypeScript**	**Бележки**
Празно множество	never	“never” не съдържа нищо освен самото себе си
Множество с един елемент	undefined / null / literal type	
Крайно множество	boolean / union	
Безкрайно множество	string / number / object	
Универсално множество	any / unknown	Всеки елемент е член на “any”, а всяко множество е негово подмножество / “unknown” е типово безопасна алтернатива на “any”

Ето няколко примера:

TypeScript	**Термин от теорията на множествата**	**Пример**
never	∅ (празно множество)	const x: never = ‘x’; // Грешка: Тип ‘string’ не е присвоим към тип ‘never’

Literal type	Множество с един елемент	type X = ‘X’; type Y = 7;
Стойност присвоима към T	Стойност ∈ T (член на)	type XY = ‘X’ | ‘Y’; const x: XY = ‘X’;
T1 присвоим към T2	T1 ⊆ T2 (подмножество на)	type XY = ‘X’ | ‘Y’; const x: XY = ‘X’; const j: XY = ‘J’; // Тип “‘J”’ не е присвоим към тип ‘XY’.
T1 extends T2	T1 ⊆ T2 (подмножество на)	type X = ‘X’ extends string ? true : false;
T1 | T2	T1 ∪ T2 (обединение)	type XY = ‘X’ | ‘Y’; type JK = 1 | 2;
T1 & T2	T1 ∩ T2 (сечение)	type X = { a: string } type Y = { b: string } type XY = X & Y const x: XY = { a: ‘a’, b: ‘b’ }
unknown	Универсално множество	const x: unknown = 1

Обединение, (T1 | T2) създава по-широко множество (и двете):

```
type X = {
    a: string;
};
type Y = {
    b: string;
};
```

```
type XY = X | Y;
const r: XY = { a: 'a', b: 'x' }; // Валидно
```

Сечение, (T1 & T2) стварае па-тонкае мноства (толькі агульныя элементы):

```
type X = {
    a: string;
};
type Y = {
    a: string;
    b: string;
};
type XY = X & Y;
const r: XY = { a: 'a' }; // Невалідна
const j: XY = { a: 'a', b: 'b' }; // Валідна
```

Ключавое слова `extends` у гэтым кантэксце можна разглядаць як “падмноства”. Яно задае абмежаванне для тыпу. Калі `extends` выкарыстоўваецца з generic, яно абмяжоўвае generic-параметр тыпу больш канкрэтным тыпам.

Звярніце ўвагу, што `extends` тут не мае нічога агульнага з наследаваннем класаў у сэнсе ААП.

TypeScript працуе са структурнымі тыпамі і не мае строгай намінальнай іерархіі. На самай справе, як у прыкладзе ніжэй, два тыпы могуць перакрывацца, пры гэтым ніводзін з іх не з’яўляецца падтыпам іншага, таму што TypeScript улічвае структуру, або форму, аб’ектаў.

```
interface X {
    a: string;
}
interface Y extends X {
```

```
    b: string;
}
interface Z extends Y {
    c: string;
}
const z: Z = { a: 'a', b: 'b', c: 'c' };
interface X1 {
    a: string;
}
interface Y1 {
    a: string;
    b: string;
}
interface Z1 {
    a: string;
    b: string;
    c: string;
}
const z1: Z1 = { a: 'a', b: 'b', c: 'c' };

const r: Z1 = z; // Валидно
```

Присвояване на тип: Декларации и проверки на типове

Тип може да бъде зададен по различни начини в TypeScript:

Декларация на тип

В следния пример използваме x: X (“: Type”), за да декларираме тип за променливата x.

```
type X = {
    a: string;
};
```

```
// Декларация на тип
const x: X = {
    a: 'a',
};
```

Ако променливата не е в посочения формат, TypeScript ще отчете грешка. Например:

```
type X = {
    a: string;
};

const x: X = {
    a: 'a',
     b: 'b', // Error: Object literal may only specify known
properties
};
```

Проверка на тип (Type Assertion)

Може да се добави проверка чрез ключовата дума `as`. Това указва на компилатора, че разработчикът разполага с повече информация за даден тип, и потиска евентуалните грешки.

Например:

```
type X = {
    a: string;
};
const x = {
    a: 'a',
    b: 'b',
} as X;
```

В горния пример обектът x е деклариран като тип X чрез използването на ключовата дума as. Това информира

TypeScript компилатора, че обектът отговаря на посочения тип, въпреки че има допълнително свойство b, което не присъства в дефиницията на типа.

Type assertions са полезни в ситуации, когато трябва да се посочи по-специфичен тип, особено при работа с DOM. Например:

```
const myInput = document.getElementById('my_input') as
HTMLInputElement;
```

Тук type assertion as HTMLInputElement се използва, за да се укаже на TypeScript, че резултатът от getElementById трябва да бъде третиран като HTMLInputElement. Type assertions могат също да се използват за пренасочване (remap) на ключове, както е показано в примера по-долу с template literals:

```
type J<Type> = {
    [Property in keyof Type as `prefix_${string &
        Property}`]: () => Type[Property];
};
type X = {
    a: string;
    b: number;
};
type Y = J<X>;
```

В този пример типът `J<Type>` използва mapped type с template literal, за да пренасочи ключовете на Type. Той създава нови свойства с добавен префикс “prefix_” към всеки ключ, като съответните им стойности са функции, които връщат оригиналните стойности на свойствата.

Струва си да се отбележи, че при използване на type assertion TypeScript няма да извърши excess property checking. Затова обикновено е за предпочитане да се използва декларация на тип, когато структурата на обекта е известна предварително.

Ambient Declarations

Ambient declarations са файлове, които описват типове за JavaScript код. Те имат файлов формат `.d.ts`. Обикновено се импортират и използват за анотиране на съществуващи JavaScript библиотеки или за добавяне на типове към съществуващи JS файлове във вашия проект.

Типовете за много често използвани библиотеки могат да бъдат намерени на: https://github.com/DefinitelyTyped/DefinitelyTyped/

и могат да бъдат инсталирани с помощта на:

```
npm install --save-dev @types/library-name
```

За вашите дефинирани Ambient Declarations, можете да ги импортирате, използвайки “triple-slash” reference:

```
/// <reference path="./library-types.d.ts" />
```

Можете да използвате Ambient Declarations дори в JavaScript файлове, като използвате `// @ts-check`.

Ключовата дума `declare` позволява дефиниране на типове за съществуващ JavaScript код, без да е необходимо да се импортира, като служи като заместител за типове от друг файл или глобално.

Проверка на свойства и проверка за излишни свойства

TypeScript се базира на структурна типова система, но проверката за излишни свойства е характеристика на TypeScript, която му позволява да проверява дали обектът има точно определените свойства, посочени в типа.

Проверката за излишни свойства се извършва например при присвояване на обектни литерали на променливи или при предаването им като аргументи на функцията за излишни свойства.

```
type X = {
    a: string;
};
const y = { a: 'a', b: 'b' };
const x: X = y; // Валидно благодарение на структурното типизиране
const w: X = { a: 'a', b: 'b' }; // Невалидно поради проверка за излишък на свойства
```

Слаби типове

Един тип се счита за слаб, когато съдържа единствено множество от изцяло опционални свойства:

```
type X = {
    a?: string;
    b?: string;
};
```

TypeScript счита за грешка присвояването на каквото и да е към слаб тип, когато няма припокриване, например следният код ще хвърли грешка:

```ts
type Options = {
    a?: string;
    b?: string;
};

const fn = (options: Options) => undefined;

fn({ c: 'c' }); // Невалидно
```

Въпреки че не се препоръчва, ако е необходимо, е възможно да се заобиколи тази проверка, като се използва type assertion:

```ts
type Options = {
    a?: string;
    b?: string;
};
const fn = (options: Options) => undefined;
fn({ c: 'c' } as Options); // Валидно
```

Или като добавите `unknown` към сигнатурата на индекса на слабия тип:

```ts
type Options = {
    [prop: string]: unknown;
    a?: string;
    b?: string;
};

const fn = (options: Options) => undefined;
fn({ c: 'c' }); // Валидно
```

Строга проверка на обектни литерали (Freshness)

Strict object literal checking, понякога наричана “freshness”, е функционалност в TypeScript, която помага да се откриват излишни или грешно изписани свойства, които иначе биха

останали незабелязани при стандартните структурни проверки на типовете.

Когато се създава обектен литерал, TypeScript компилаторът го счита за “fresh”. Ако обектният литерал бъде присвоен на променлива или подаден като параметър, TypeScript ще върне грешка, ако са зададени свойства, които не съществуват в целевия тип.

Въпреки това “freshness” изчезва, когато обектният литерал бъде разширен (widened) или когато се използва type assertion.

Ето няколко примера за илюстрация:

```
type X = { a: string };
type Y = { a: string; b: string };

let x: X;
x = { a: 'a', b: 'b' }; // Проверка за валидност: Невалидно присвояване
var y: Y;
y = { a: 'a', bx: 'bx' }; // Проверка за валидност: Невалидно присвояване

const fn = (x: X) => console.log(x.a);

fn(x);
fn(y); // Разширяване: Без грешки, съвместимост на типовете по структура

fn({ a: 'a', bx: 'b' }); // Проверка за валидност: Невалидно присвояване

let c: X = { a: 'a' };
let d: Y = { a: 'a', b: '' };
c = d; // Разширяване: Без проверка за валидност
```

Извеждане на типове

TypeScript може да извежда типове, когато не е предоставена анотация при:

- Инициализация на променлива.
- Инициализация на член (member).
- Задаване на стойности по подразбиране за параметри.
- Типа на връщаната стойност от функция.

Например:

```
let x = 'x'; // Типът, който се извежда, е string
```

Компилаторът на TypeScript анализира стойността или израза и определя неговия тип въз основа на наличната информация.

По-сложни изводи

Когато при извеждането на типове се използват няколко израза, TypeScript търси “най-добрите общи типове”. Например:

```
let x = [1, 'x', 1, null]; // Типът, който се извежда, е (string | number | null)[]
```

Ако компилаторът не успее да намери най-подходящите общи типове, той връща тип “union”. Например:

```
let x = [new RegExp('x'), new Date()]; // Типът, който се извежда, е (RegExp | Date)[]
```

TypeScript използва “contextual typing”, базирано на местоположението на променливата, за да изведе

типовете. В следния пример компилаторът знае, че `e` е от тип `MouseEvent`, благодарение на типа на събитието `click`, дефиниран във файла lib.d.ts, който съдържа общи декларации за различни често срещани конструкции в JavaScript и DOM:

```
window.addEventListener('click', function (e) {}); // Типът, който се извежда за "e" е "MouseEvent"
```

Разширяване на типовете

Разширяването на типовете е процесът, при който TypeScript присвоява тип на инициализирана променлива, когато не е зададена типова анотация. То позволява преминаване от по-тесен към по-широк тип, но не и обратното. В следния пример:

```
let x = 'x'; // TypeScript извежда като string, широк тип
let y: 'y' | 'x' = 'y'; // Типът на y е "union" на литерални типове
y = x; // Невалиден Тип 'string' не може да бъде присвоен на типа '"x" | "y"'.
```

TypeScript присвоява `string` на `x` въз основа на единствената стойност, предоставена по време на инициализацията (`x`), това е пример за разширяване.

TypeScript предоставя начини за контрол на процеса на разширяване, например чрез използване на “const”.

Const

Използването на ключовата дума `const` при деклариране на променлива води до по-тясно извеждане на типове в TypeScript.

Например:

```
const x = 'x'; // TypeScript извежда типа на x като 'x', по-тесен тип
let y: 'y' | 'x' = 'y';
y = x; // Валидно: Типът на x е изведен като 'x'
```

Чрез използването на `const` за деклариране на променливата x, нейният тип се стеснява до конкретната литерална стойност ‘x’. Тъй като типът на x е по-тесен, той може да бъде присвоен на променливата y без грешка. Причината е, че `const` променливите не могат да бъдат преназначавани, затова техният тип може да бъде стеснен до конкретен литерален тип — в този случай ‘x’.

Const Modifier on Type Parameters

От версия 5.0 на TypeScript е възможно да се зададе `const` атрибут върху generic параметър. Това позволява извеждане на възможно най-точния тип. Нека видим пример без използване на `const`:

```
function identity<T>(value: T) {
    // Няма const тук
    return value;
}
const values = identity({ a: 'a', b: 'b' }); // Изведения тип е: { a: string; b: string; }
```

Както се вижда, свойствата `a` и `b` са изведени като тип `string`.

Сега нека видим разликата при използване на `const`:

```
function identity<const T>(value: T) {
    // Използване на const modifier върху type параметри
```

```
    return value;
}
const values = identity({ a: 'a', b: 'b' }); // Изведения тип
e: { a: "a"; b: "b"; }
```

Тук свойствата `a` и `b` са изведени като `const`, т.е. се третират като string литерали, а не просто като тип `string`.

Const assertion

Тази функционалност позволява да декларирате променлива с по-прецизен литерален тип въз основа на нейната начална стойност, като указва на компилатора, че стойността трябва да се третира като неизменим литерал. Ето няколко примера:

За отделно свойство:

```
const v = {
    x: 3 as const,
};
v.x = 3;
```

За цял обект:

```
const v = {
    x: 1,
    y: 2,
} as const;
```

Това е особено полезно при дефиниране на тип за tuple:

```
const x = [1, 2, 3]; // number[]
const y = [1, 2, 3] as const; // Tuple от readonly [1, 2, 3]
```

Явна типова анотация

Можем изрично да зададем тип. В следния пример свойството `x` е от тип `number`:

```
const v = {
    x: 1, // Предполагаем тип: число (разширяване)
};
v.x = 3; // Валидно
```

Можем да направим типовата анотация по-конкретна, като използваме обединение на литерални типове:

```
const v: { x: 1 | 2 | 3 } = {
    x: 1, // x вече е обединение от литерални типове: 1 | 2 | 3
};
v.x = 3; // Валидно
v.x = 100; // Невалидно
```

Стесняване на типове

Стесняването на типове е процес в TypeScript, при който един по-общ тип се стеснява до по-специфичен. Това се случва, когато TypeScript анализира кода и установи, че определени условия или операции могат да уточнят типовата информация.

Стесняването на типове може да се случи по различни начини, включително:

Условия

Чрез използване на условни конструкции, като `if` или `switch`, TypeScript може да стесни типа въз основа на резултата от условието. Например:

```
let x: number | undefined = 10;

if (x !== undefined) {
      x += 100; // Типът е число, което е било ограничено от условието
}
```

Изключване на грешка или връщане от разклонение

Изключването на грешка или преждевременното връщане от разклонение може да се използва, за да помогне на TypeScript да стесни типа. Например:

```
let x: number | undefined = 10;

if (x === undefined) {
    throw 'error';
}
x += 100;
```

Други начини за стесняване на типове в TypeScript включват:

- операторът `instanceof`: използва се за проверка дали даден обект е инстанция на конкретен клас.
- операторът `in`: използва се за проверка дали дадено свойство съществува в обект.
- операторът `typeof`: използва се за проверка на типа на стойност по време на изпълнение.
- вградени функции като `Array.isArray()`: използват се за проверка дали дадена стойност е масив.

Discriminated Union

Използването на “Discriminated Union” е патърн в TypeScript, при който се добавя изричен “таг” към обектите, за да се разграничат различните типове в едно обединение. Този патърн се нарича още “tagged union”. В следния пример “тагът” е представен чрез свойството “type”:

```
type A = { type: 'type_a'; value: number };
type B = { type: 'type_b'; value: string };

const x = (input: A | B): string | number => {
    switch (input.type) {
        case 'type_a':
            return input.value + 100; // типът е A
        case 'type_b':
            return input.value + 'extra'; // типът е B
    }
};
```

Потребителски type guards

В случаите, когато TypeScript не може да определи тип, е възможно да се напише помощна функция, известна като “user-defined type guard”. В следния пример ще използваме type predicate, за да стесним типа след прилагане на определено филтриране:

```
const data = ['a', null, 'c', 'd', null, 'f'];

const r1 = data.filter(x => x != null); // Типът е (string | null)[], TypeScript не успя да определи типа правилно

const isValid = (item: string | null): item is string => item !== null; // Потребителски type guard
```

```
const r2 = data.filter(isValid); // Типът вече е string[], като
използвахме predicate type guard успяхме да стесним типа
```

Switch-true narrowing

TypeScript 5.3 добавя switch-true narrowing, което ви позволява да замените сложни if/else вериги с switch (true), използвайки булеви условия. Това подобрява четимостта и все още стеснява типовете. Подобно е на pattern matching, но по-просто.

```
function classify(x: unknown) {
    switch (true) {
        case typeof x === 'string':
            return `"${x.toUpperCase()}"`;
        case typeof x === 'number':
            return x > 0 ? 'positive' : 'negative';
        case Array.isArray(x):
            return `[${x.length} items]`;
        default:
            return 'something else';
    }
}
```

Примитивни типове

TypeScript поддържа 7 примитивни типа. Примитивният тип данни е тип, който не е обект и няма свързани с него методи. В TypeScript всички примитивни типове са неизменни, което означава, че стойностите им не могат да бъдат променяни, след като бъдат присвоени.

string

Примитивният тип `string` съхранява текстови данни, като стойността винаги се поставя в двойни или единични кавички.

```
const x: string = 'x';
const y: string = 'y';
```

Низовете могат да обхващат няколко реда, ако са обградени със символа обратна кавичка (`):

```
let sentence: string = `xxx,
   yyy`;
```

boolean

Типът данни `boolean` в TypeScript съхранява двоична стойност, която може да бъде `true` или `false`.

```
const isReady: boolean = true;
```

number

Типът данни `number` в TypeScript се представя като 64-битова стойност с плаваща запетая. Типът `number` може да представя цели числа и дроби. TypeScript поддържа също шестнадесетични, двоични и осмични числа, например:

```
const decimal: number = 10;
const hexadecimal: number = 0xa00d; // Шестнадесетичната система започва с 0x
const binary: number = 0b1010; // Двоичната система започва с 0b
const octal: number = 0o633; // Осмичната система започва с 0o
```

bigInt

`bigInt` представлява числови стойности, които са много големи (253 – 1) и не могат да бъдат представени с `number`.

`bigInt` може да бъде създаден чрез извикване на вградената функция `BigInt()` или чрез добавяне на `n` в края на всеки целочислен литерал:

```
const x: bigint = BigInt(9007199254740991);
const y: bigint = 9007199254740991n;
```

Забележки:

- Стойностите от тип `bigInt` не могат да се смесват със стойности от тип `number` и не могат да се използват с вградената библиотека `Math`; те трябва да бъдат преобразувани към един и същ тип.
- Стойностите от тип `bigInt` са достъпни само ако целевата конфигурация е ES2020 или по-нова.

Символ

Символите са уникални идентификатори, които могат да се използват като ключове на свойства в обекти, за да се предотвратят конфликти при наименуване.

```
type Obj = {
    [sym: symbol]: number;
};

const a = Symbol('a');
const b = Symbol('b');
let obj: Obj = {};
obj[a] = 123;
obj[b] = 456;
```

```
console.log(obj[a]); // 123
console.log(obj[b]); // 456
```

null и undefined

Типовете `null` и `undefined` представляват липса на стойност или отсъствие на каквато и да е стойност.

Типът `undefined` означава, че стойността не е присвоена или инициализирана, или показва непреднамерено отсъствие на стойност.

Типът `null` означава, че знаем, че полето няма стойност, така че стойността не е налична, което показва умишлено отсъствие на стойност.

Масив

`array` е тип данни, който може да съхранява множество стойности от един и същ тип или не. Той може да бъде дефиниран, като се използва следният синтаксис:

```
const x: string[] = ['a', 'b'];
const y: Array<string> = ['a', 'b'];
const j: Array<string | number> = ['a', 1, 'b', 2]; // Union
```

TypeScript поддържа readonly масиви чрез следния синтаксис:

```
const x: readonly string[] = ['a', 'b']; // Readonly модификатор
const y: ReadonlyArray<string> = ['a', 'b'];
const j: ReadonlyArray<string | number> = ['a', 1, 'b', 2];
j.push('x'); // Невалидно
```

TypeScript поддържа tuple и readonly tuple:

```
const x: [string, number] = ['a', 1];
const y: readonly [string, number] = ['a', 1];
```

any

Типът `any` представлява буквално “всякаква” стойност и е стойността по подразбиране, когато TypeScript не може да изведе тип или когато такъв не е указан.

При използване на `any` TypeScript компилаторът пропуска проверката на типовете, което означава, че няма типова безопасност. По принцип не бива да се използва `any`, за да се заглуши компилаторът при възникване на грешка. Вместо това е по-добре грешката да бъде отстранена, тъй като чрез използването на `any` могат да се нарушат контрактите и се губят предимствата на TypeScript като autocomplete.

Типът `any` може да бъде полезен по време на постепенна миграция от JavaScript към TypeScript, тъй като може временно да заглуши компилатора.

При нови проекти използвайте TypeScript конфигурацията `noImplicitAny`, която позволява на TypeScript да генерира грешки, когато `any` се използва или бъде изведен автоматично.

Типът `any` често е източник на грешки, тъй като може да прикрие реални проблеми с типовете. Избягвайте използването му възможно най-много.

Анотации на типовете

При променливи, декларирани с `var`, `let` и `const`, по желание може да бъде добавена типова анотация:

```
const x: number = 1;
```

TypeScript се справя добре с извеждането на типове, особено когато са прости, така че тези декларации в повечето случаи не са необходими.

При функции е възможно да се добавят анотации на типовете към параметрите:

```
function sum(a: number, b: number) {
    return a + b;
}
```

Ето един пример с използване на анонимни функции (т.нар. lambda функции):

```
const sum = (a: number, b: number) => a + b;
```

Тези анотации могат да бъдат избегнати, когато за параметър е зададена стойност по подразбиране:

```
const sum = (a = 10, b: number) => a + b;
```

Към функциите могат да се добавят анотации за типа на връщаната стойност:

```
const sum = (a = 10, b: number): number => a + b;
```

Това е полезно особено за по-сложни функции, тъй като написването на явен тип на връщаната стойност преди имплементацията може да помогне за по-добро обмисляне на функцията.

Като цяло е препоръчително да се посочват типовете на сигнатурите, но не и на локалните променливи в тялото на функцията, а типовете винаги да се добавят към литералите на обектите.

Опционални свойства

Един обект може да дефинира опционални свойства, като добави въпросителен знак `?` в края на името на свойствата:

```
type X = {
    a: number;
    b?: number; // Опционално свойство
};
```

Възможно е да се зададе стойност по подразбиране, когато свойството е опционално:

```
type X = {
    a: number;
    b?: number;
};
const x = ({ a, b = 100 }: X) => a + b;
```

Readonly свойства

Възможно е да се предотврати записването в дадено свойство чрез използването на модификатора `readonly`, който гарантира, че свойството не може да бъде презаписвано, но не предоставя пълна гаранция за цялостна неизменяемост:

```
interface Y {
    readonly a: number;
}
```

```
type X = {
    readonly a: number;
};

type J = Readonly<{
    a: number;
}>;

type K = {
    readonly [index: number]: string;
};
```

Сигнатури на индекси

В TypeScript като сигнатури на индекси можем да използваме `string`, `number` и `symbol`:

```
type K = {
    [name: string | number]: string;
};
const k: K = { x: 'x', 1: 'b' };
console.log(k['x']);
console.log(k[1]);
console.log(k['1']); // Същият резултат като k[1]
```

Моля, обърнете внимание, че JavaScript автоматично преобразува индекс с `number` в индекс с `string`, така че `k[1]` или `k["1"]` връщат същата стойност.

Разширяване на типове

Възможно е да се разшири `interface` (да се копират елементи от друг тип):

```
interface X {
    a: string;
}
interface Y extends X {
    b: string;
}
```

Възможно е също така да се разшири от няколко типа:

```
interface A {
    a: string;
}
interface B {
    b: string;
}
interface Y extends A, B {
    y: string;
}
```

Ключовата дума `extends` работи само с interfaces и класове; при types се използва сечение (intersection):

```
type A = {
    a: number;
};
type B = {
    b: number;
};
type C = A & B;
```

Тып можна пашырыць з дапамогай інтэрфейсу, але не наадварот:

```
type A = {
    a: string;
};
interface B extends A {
```

```
    b: string;
}
```

Literal Types

Literal type е множество с един елемент от по-общ тип, което дефинира много конкретна стойност, представляваща JavaScript примитив.

Literal types в TypeScript са числа, низове и булеви стойности.

Пример за literal стойности:

```
const a = 'a'; // String literal type
const b = 1; // Numeric literal type
const c = true; // Boolean literal type
```

String, Numeric и Boolean literal types се използват в unions, type guards и type aliases. В следния пример е показан union type alias. `O` се състои само от посочените стойности — други низове не са валидни:

```
type O = 'a' | 'b' | 'c';
```

Literal Inference

Literal inference е функционалност в TypeScript, която позволява типът на променлива или параметър да бъде изведен въз основа на стойността му.

В следния пример може да се види, че TypeScript счита `x` за literal type, тъй като стойността не може да бъде променяна

по-късно, докато `y` се извежда като string, тъй като може да бъде променяна:

```
const x = 'x'; // Типът на 'x' е literal type, тъй като тази стойност не може да бъде променена
let y = 'y'; // Типът е string, тъй като тази стойност може да бъде променена
```

В следния пример може да се види, че `o.x` е изведено като `string` (а не като literal на `a`), тъй като TypeScript счита, че стойността може да бъде променяна по-късно.

```
type X = 'a' | 'b';

let o = {
    x: 'a', // Това е по-широк тип string
};

const fn = (x: X) => `${x}-foo`;

console.log(fn(o.x)); // Argument of type 'string' is not assignable to parameter of type 'X'
```

Както се вижда, кодът хвърля грешка при подаване на `o.x` към `fn`, тъй като `X` е по-тесен тип.

Можем да решим този проблем чрез използване на type assertion с `const` или чрез типа `X`:

```
let o = {
    x: 'a' as const,
};
```

или:

```
let o = {
    x: 'a' as X,
```

```
};
```

strictNullChecks

`strictNullChecks` е опция на TypeScript компилатора, която налага стриктна проверка за null стойности. Когато тази опция е активирана, променливи и параметри могат да получават стойности `null` или `undefined` само ако изрично са декларирани като такива чрез union типа `null | undefined`. Ако променлива или параметър не е изрично деклариран като nullable, TypeScript ще генерира грешка, за да предотврати потенциални по време на изпълнение грешки.

Enums

В TypeScript `enum` представлява множество от именувани константни стойности.

```
enum Color {
    Red = '#ff0000',
    Green = '#00ff00',
    Blue = '#0000ff',
}
```

Enums могат да бъдат дефинирани по различни начини:

Числови enums

В TypeScript, Numeric Enum е enum, при който всяка константа получава числова стойност, започвайки от 0 по подразбиране.

```
enum Size {
    Small, // стойността започва от 0
    Medium,
    Large,
}
```

Възможно е да се зададат потребителски стойности чрез изричното им присвояване:

```
enum Size {
    Small = 10,
    Medium,
    Large,
}
console.log(Size.Medium); // 11
```

Низови enums

В TypeScript, низовият enum е Enum, при който всяка константа получава низова стойност.

```
enum Language {
    English = 'EN',
    Spanish = 'ES',
}
```

Забележка: TypeScript позволява използването на хетерогенни enums, където низови и числови членове могат да съществуват заедно.

Константни enums

Константният enum в TypeScript е специален вид Enum, при който всички стойности са известни още при компилирането и се вмъкват директно във всеки случай, в който се използва enum, което води до по-ефективен код.

```
const enum Language {
    English = 'EN',
    Spanish = 'ES',
}
console.log(Language.English);
```

Ще бъде компилирано в:

```
console.log('EN' /* Language.English */);
```

Забележка: константните enums имат предварително зададени стойности, като самият enum се премахва при компилация, което може да е по-ефективно в самостоятелни библиотеки, но обикновено не е желателно. Също така, константните enums не могат да имат изчисляеми членове.

Обратни съпоставки

В TypeScript, обратните съпоставки в Enums се отнасят до възможността да се извлече името на член на Enum от неговата стойност. По подразбиране членовете на Enum имат директни съпоставки от име към стойност, но обратни съпоставки могат да се създадат чрез явно задаване на стойности за всеки член. Обратните съпоставки са полезни, когато трябва да намерите член на Enum по стойността му или когато трябва да итерирате през всички членове на Enum. Обърнете внимание, че само числовите членове на Enum генерират обратни съпоставки, докато членовете на String Enum изобщо не получават генерирана обратна съпоставка.

Следният enum:

```
enum Grade {
    A = 90,
    B = 80,
    C = 70,
    F = 'fail',
}
```

Компилира се в:

```
'use strict';
var Grade;
(function (Grade) {
    Grade[(Grade['A'] = 90)] = 'A';
    Grade[(Grade['B'] = 80)] = 'B';
    Grade[(Grade['C'] = 70)] = 'C';
    Grade['F'] = 'fail';
})(Grade || (Grade = {}));
```

Следователно, съпоставянето на стойности към ключове работи за числовите членове на enum, но не и за низовите членове на enum:

```
enum Grade {
    A = 90,
    B = 80,
    C = 70,
    F = 'fail',
}
const myGrade = Grade.A;
console.log(Grade[myGrade]); // A
console.log(Grade[90]); // A

const failGrade = Grade.F;
console.log(failGrade); // fail
console.log(Grade[failGrade]);  //  Element  implicitly  has  an
'any' type because index expression is not of type 'number'.
```

Ambient enums

Ambient enum в TypeScript е тип Enum, който е дефиниран в декларационен файл (*.d.ts) без свързана реализация. Той позволява да се дефинира набор от именовани константи, които могат да се използват по тип-безопасен начин в различни файлове, без да е необходимо да се импортират детайлите на реализацията във всеки файл.

Изчисляеми и константни членове

В TypeScript, изчисляем член е член на Enum, чиито стойност се изчислява по време на изпълнение, докато константен член е член, чиито стойност е зададена по време на компилация и не може да се промени по време на изпълнение. Изчисляемите членове са разрешени в обикновени Enums, докато константните членове са разрешени както в обикновени, така и в константни enums.

```
// Константни членове
enum Color {
    Red = 1,
    Green = 5,
    Blue = Red + Green,
}
console.log(Color.Blue); // 6 генерирано по време на компилация

// Изчисляеми членове
enum Color {
    Red = 1,
    Green = Math.pow(2, 2),
    Blue = Math.floor(Math.random() * 3) + 1,
}
console.log(Color.Blue); // случайно число, генерирано по време
на изпълнение
```

Enums се означават чрез unions, съставени от типовете на техните членове. Стойностите на всеки член могат да се

определят чрез константни или неконстантни изрази, като членовете с константни стойности се присвояват на литерални типове. За илюстрация, разгледайте декларацията на type E и неговите подтипове E.A, E.B и E.C. В този случай E представлява union E.A | E.B | E.C.

```
const identity = (value: number) => value;

enum E {
    A = 2 * 5, // Числов литерал
    B = 'bar', // Низов литерал
    C = identity(42), // Изчисляем член
}

console.log(E.C); //42
```

Narrowing

TypeScript narrowing е процес на уточняване на типа на променлива в рамките на условен блок. Това е полезно при работа с union типове, където променлива може да има повече от един тип.

TypeScript поддържа няколко начина за стесняване на типа:

Проверки за типа “typeof”

Проверката за типа “typeof” е специфична проверка в TypeScript, която проверява типа на променлива въз основа на нейния вграден тип в JavaScript.

```
const fn = (x: number | string) => {
    if (typeof x === 'number') {
        return x + 1; // x е число
```

```
    }
    return -1;
};
```

Свиване на тип чрез truthiness

Свиването на тип чрез truthiness в TypeScript работи като проверява дали променлива е truthy или falsy, за да стесни съответно нейния тип.

```
const toUpperCase = (name: string | null) => {
    if (name) {
        return name.toUpperCase();
    } else {
        return null;
    }
};
```

Свиване на тип чрез равенство

Свиването на тип чрез равенство в TypeScript работи чрез проверка дали променлива е равна на конкретна стойност или не, за да се стесни съответно нейният тип.

Използва се в комбинация с `switch` изрази и оператори за равенство като `===`, `!==`, `==` и `!=`, за да се стесни типът.

```
const checkStatus = (status: 'success' | 'error') => {
    switch (status) {
        case 'success':
            return true;
        case 'error':
            return null;
    }
};
```

Свиване на тип чрез оператора “in”

Свиването на тип чрез оператора `in` в TypeScript е начин да се стесни типът на променлива, базирайки се на това дали дадено свойство съществува в типа на променливата.

```
type Dog = {
    name: string;
    breed: string;
};

type Cat = {
    name: string;
    likesCream: boolean;
};

const getAnimalType = (pet: Dog | Cat) => {
    if ('breed' in pet) {
        return 'dog';
    } else {
        return 'cat';
    }
};
```

Свиване на тип чрез `instanceof`

Свиването на тип чрез оператора `instanceof` в TypeScript е начин да се стесни типът на променлива, базирайки се на нейната конструкторска функция, чрез проверка дали обектът е инстанция на определен клас или interface.

```
class Square {
    constructor(public width: number) {}
}
class Rectangle {
    constructor(
        public width: number,
```

```
        public height: number
    ) {}
}
function area(shape: Square | Rectangle) {
    if (shape instanceof Square) {
        return shape.width * shape.width;
    } else {
        return shape.width * shape.height;
    }
}
const square = new Square(5);
const rectangle = new Rectangle(5, 10);
console.log(area(square)); // 25
console.log(area(rectangle)); // 50
```

Присвоявания

Свиването на типове в TypeScript чрез присвоявания е начин да се стесни типът на променлива въз основа на стойността, която ѝ е присвоена. Когато на променлива се присвои стойност, TypeScript извежда нейния тип въз основа на присвоената стойност и стеснява типа на променливата, за да съответства на изведения тип.

```
let value: string | number;
value = 'hello';
if (typeof value === 'string') {
    console.log(value.toUpperCase());
}
value = 42;
if (typeof value === 'number') {
    console.log(value.toFixed(2));
}
```

Анализ на потока на управление

Анализът на потока на управление в TypeScript е начин за статичен анализ на потока на кода с цел извеждане на типовете на променливите, което позволява на компилатора да стеснява типовете на тези променливи при необходимост, базирайки се на резултатите от анализа.

Преди TypeScript 4.4, анализът на потока на кода се прилагаше само за код в рамките на if изрази, но от TypeScript 4.4 нататък той може да се прилага и за условни изрази и достъпи до дискриминантни свойства, косвено реферирани чрез const променливи.

Например:

```
const f1 = (x: unknown) => {
    const isString = typeof x === 'string';
    if (isString) {
        x.length;
    }
};

const f2 = (
     obj: { kind: 'foo'; foo: string } | { kind: 'bar'; bar:
number }
) => {
    const isFoo = obj.kind === 'foo';
    if (isFoo) {
        obj.foo;
    } else {
        obj.bar;
    }
};
```

Някои примери, при които не се наблюдава стесняване:

```
const f1 = (x: unknown) => {
    let isString = typeof x === 'string';
    if (isString) {
          x.length; // Грешка, няма стесняване, защото isString
не е const
    }
};

const f6 = (
     obj: { kind: 'foo'; foo: string } | { kind: 'bar'; bar:
number }
) => {
    const isFoo = obj.kind === 'foo';
    obj = obj;
    if (isFoo) {
          obj.foo; // Грешка: няма стесняване, тъй като obj се
присвоява в тялото на функцията
    }
};
```

Забележка: В условните изрази TypeScript може да анализира до пет нива на индирекция.

Type Predicates

Type Predicates в TypeScript са функции, които връщат boolean стойност и се използват за стесняване на типа на променлива до по-специфичен тип.

```
const isString = (value: unknown): value is string => typeof
value === 'string';

const foo = (bar: unknown) => {
    if (isString(bar)) {
        console.log(bar.toUpperCase());
    } else {
        console.log('не е низ');
```

```
    }
};
```

TypeScript 5.5 автоматично извежда type predicates (като `x is T`) във функции като `.filter`, така че знае кога стойности като undefined са премахнати—давайки по-точни типове и по-малко грешки; това работи за ясни проверки (например `x !== undefined`), но не и за двусмислени като `!!x`.

```
const nums = [1, null, 2].filter(x => x !== null);
```

Discriminated Unions

Discriminated Unions в TypeScript са тип на обединение, който използва общо свойство, известно като дискриминант, за да стесни набора от възможни типове за обединението.

```
type Square = {
    kind: 'square'; // Дискриминант
    size: number;
};

type Circle = {
    kind: 'circle'; // Дискриминант
    radius: number;
};

type Shape = Square | Circle;

const area = (shape: Shape) => {
    switch (shape.kind) {
        case 'square':
            return Math.pow(shape.size, 2);
        case 'circle':
            return Math.PI * Math.pow(shape.radius, 2);
```

```
    }
};

const square: Square = { kind: 'square', size: 5 };
const circle: Circle = { kind: 'circle', radius: 2 };

console.log(area(square)); // 25
console.log(area(circle)); // 12.566370614359172
```

The never Type

Когато променлива бъде стеснена до тип, който не може да съдържа никакви стойности, компилаторът на TypeScript ще извлече, че променливата трябва да бъде от тип `never`. Това е така, защото типът `never` представлява стойност, която никога не може да бъде произведена.

```
const printValue = (val: string | number) => {
    if (typeof val === 'string') {
        console.log(val.toUpperCase());
    } else if (typeof val === 'number') {
        console.log(val.toFixed(2));
    } else {
        // val има тип never тук, защото не може да бъде нищо
друго освен низ или число
        const neverVal: never = val;
        console.log(`Unexpected value: ${neverVal}`);
    }
};
```

Проверка за изчерпателност

Проверката за изчерпателност е функция в TypeScript, която гарантира, че всички възможни случаи на дискриминиран union са обработени в `switch` или `if` израз.

```
type Direction = 'up' | 'down';

const move = (direction: Direction) => {
    switch (direction) {
        case 'up':
            console.log('Moving up');
            break;
        case 'down':
            console.log('Moving down');
            break;
        default:
            const exhaustiveCheck: never = direction;
               console.log(exhaustiveCheck); // Този ред никога
няма да бъде изпълнен
    }
};
```

Типът `never` се използва, за да се гарантира, че default случаят е изчерпателен и че TypeScript ще генерира грешка, ако бъде добавена нова стойност към типа Direction без да бъде обработена в switch израза.

Обектни типове

В TypeScript, обектните типове описват структурата на един обект. Те задават имената и типовете на свойствата на обекта, както и дали тези свойства са задължителни или по избор.

В TypeScript можете да дефинирате обектни типове по два основни начина:

Interface, който определя структурата на обект, като задава имената, типовете и опционалността на неговите свойства.

```
interface User {
    name: string;
    age: number;
    email?: string;
}
```

Type alias, подобно на interface, определя структурата на обект. Въпреки това, той може също така да създаде нов персонализиран тип, базиран на съществуващ тип или комбинация от съществуващи типове. Това включва дефиниране на union типове, intersection типове и други сложни типове.

```
type Point = {
    x: number;
    y: number;
};
```

Възможно е също така да се дефинира тип анонимно:

```
const sum = (x: { a: number; b: number }) => x.a + x.b;
console.log(sum({ a: 5, b: 1 }));
```

Tuple тип (анонимен)

Tuple тип е тип, който представлява масив с фиксиран брой елементи и съответните им типове. Tuple типът налага конкретен брой елементи и съответните им типове в определен ред. Tuple типовете са полезни, когато искате да представите колекция от стойности със специфични типове, при която позицията на всеки елемент в масива има конкретно значение.

```
type Point = [number, number];
```

Именуван Tuple тип (с етикети)

Tuple типовете могат да включват опционални етикети или имена за всеки елемент. Тези етикети са предназначени за по-добра четимост и помощ от инструментите и не влияят на операциите, които могат да се извършват с тях.

```
type T = string;
type Tuple1 = [T, T];
type Tuple2 = [a: T, b: T];
type Tuple3 = [a: T, T]; // Именуван Tuple тип плюс анонимен Tuple тип
```

Tuple с фиксирана дължина

Tuple с фиксирана дължина е специфичен тип tuple, който налага фиксиран брой елементи от определени типове и не позволява промени в дължината на tuple след като бъде дефиниран.

Tuple с фиксирана дължина са полезни, когато трябва да представите колекция от стойности с конкретен брой елементи и конкретни типове и искате да гарантирате, че дължината и типовете на tuple не могат да бъдат променени неволно.

```
const x = [10, 'hello'] as const;
x.push(2); // Error
```

Union тип

Union тип е тип, който представлява стойност, която може да бъде от няколко различни типа. Union типовете се

обозначават със символа | между всеки възможен тип.

```
let x: string | number;
x = 'hello'; // Валидно
x = 123; // Валидно
```

Intersection типове

Intersection тип е тип, който представлява стойност, която има всички свойства на два или повече типа. Intersection типовете се обозначават със символа & между всеки тип.

```
type X = {
    a: string;
};

type Y = {
    b: string;
};

type J = X & Y; // Intersection

const j: J = {
    a: 'a',
    b: 'b',
};
```

Индексиране на тип

Индексирането на тип се отнася до възможността да се дефинират типове, които могат да бъдат индексирани чрез ключ, който не е известен предварително, като се използва index signature за определяне на типа на свойства, които не са изрично декларирани.

```
type Dictionary<T> = {
    [key: string]: T;
};
const myDict: Dictionary<string> = { a: 'a', b: 'b' };
console.log(myDict['a']); // Връща a
```

Тип от стойност

Type from Value в TypeScript се отнася до автоматичното извеждане на тип от стойност или израз чрез type inference.

```
const x = 'x'; // TypeScript извежда 'x' като string literal с 'const' (immutable), но го разширява до 'string' с 'let' (reassignable).
```

Тип от резултат на функция

Type from Func Return се отнася до възможността автоматично да се извежда типът на стойността, върната от функция, въз основа на нейната имплементация. Това позволява на TypeScript да определи типа на върнатата стойност без изрични type анотации.

```
const add = (x: number, y: number) => x + y; // TypeScript може да изведе, че върнатият тип на функцията е number
```

Тип от модул

Type from Module се отнася до възможността да се използват експорт-натите стойности от даден модул, за да се извлекат автоматично техните типове. Когато модул експорт-ва стойност с конкретен тип, TypeScript може да използва тази информация, за да извлече автоматично типа на тази стойност при импортиране в друг модул.

```
// calc.ts
export const add = (x: number, y: number) => x + y;
// index.ts
import { add } from 'calc';
const r = add(1, 2); // r е число
```

Mapped типове

Mapped типовете в TypeScript позволяват създаване на нови типове на базата на съществуващ тип чрез трансформиране на всяко свойство с помощта на mapping функция. Чрез mapping на съществуващи типове могат да се създадат нови типове, които представят същата информация в различен формат. За създаване на mapped тип се достъпват свойствата на съществуващ тип чрез оператора `keyof`, след което те се променят, за да се създаде нов тип. В следния пример:

```
type MyMappedType<T> = {
    [P in keyof T]: T[P][];
};
type MyType = {
    foo: string;
    bar: number;
};
type MyNewType = MyMappedType<MyType>;
const x: MyNewType = {
    foo: ['hello', 'world'],
    bar: [1, 2, 3],
};
```

дефинираме MyMappedType, който обхожда свойствата на T и създава нов тип, при който всяко свойство е масив от оригиналния си тип. По този начин създаваме MyNewType, който представя същата информация като MyType, но с всяко свойство като масив.

Модификатори на Mapped типове

Модификаторите на Mapped типове в TypeScript позволяват трансформиране на свойства в рамките на съществуващ тип:

- `readonly` или `+readonly`: Прави свойството в mapped типа само за четене.
- `-readonly`: Позволява свойството в mapped типа да бъде променяемо.
- `?`: Определя свойството в mapped типа като опционално.

Примери:

```
type ReadOnly<T> = { readonly [P in keyof T]: T[P] }; // Всички свойства са маркирани като само за четене

type Mutable<T> = { -readonly [P in keyof T]: T[P] }; // Всички свойства са маркирани като променяеми

type MyPartial<T> = { [P in keyof T]?: T[P] }; // Всички свойства са маркирани като опционални
```

Conditional типове

Conditional типовете са начин за създаване на тип, който зависи от условие, като създаваният тип се определя въз основа на резултата от условието. Те се дефинират чрез ключовата дума `extends` и тернарен оператор за условен избор между два типа.

```
type IsArray<T> = T extends any[] ? true : false;

const myArray = [1, 2, 3];
const myNumber = 42;
```

```
type IsMyArrayAnArray = IsArray<typeof myArray>; // Тип true
type IsMyNumberAnArray = IsArray<typeof myNumber>; // Тип false
```

Дистрибутивни Conditional типове

Дистрибутивните Conditional типове са функционалност, която позволява даден тип да бъде разпределен върху union от типове чрез прилагане на трансформация върху всеки член на union поотделно. Това е особено полезно при работа с mapped типове или по-високоабстрактни (higher-order) типове.

```
type Nullable<T> = T extends any ? T | null : never;
type NumberOrBool = number | boolean;
type NullableNumberOrBool = Nullable<NumberOrBool>; // number |
boolean | null
```

infer извеждане на тип в Conditional типове

Ключовата дума `infer` се използва в conditional типове, за да се извлече типът на generic параметър от тип, който зависи от него. Това позволява създаването на по-гъвкави и преизползваеми дефиниции на типове.

```
type ElementType<T> = T extends (infer U)[] ? U : never;
type Numbers = ElementType<number[]>; // number
type Strings = ElementType<string[]>; // string
```

Предефинирани Conditional типове

В TypeScript, предефинираните Conditional типове са вградени conditional типове, предоставени от езика. Те са създадени, за да извършват често използвани

трансформации на типове въз основа на характеристиките на даден тип.

`Exclude<UnionType, ExcludedType>`: Премахва всички типове от UnionType, които могат да бъдат присвоени на ExcludedType.

`Extract<Type, Union>`: Извлича всички типове от Union, които могат да бъдат присвоени на Type.

`NonNullable<Type>`: Премахва null и undefined от Type.

`ReturnType<Type>`: Извлича типа на върнатата стойност на функция Type.

`Parameters<Type>`: Извлича типовете на параметрите на функция Type.

`Required<Type>`: Прави всички свойства в Type задължителни.

`Partial<Type>`: Прави всички свойства в Type опционални.

`Readonly<Type>`: Прави всички свойства в Type само за четене.

Template Union типове

Template union типовете могат да се използват за комбиниране и манипулиране на текст в рамките на type системата, например:

```
type Status = 'active' | 'inactive';
type Products = 'p1' | 'p2';
type ProductId = `id-${Products}-${Status}`; // "id-p1-active"
| "id-p1-inactive" | "id-p2-active" | "id-p2-inactive"
```

Any тип

Типът `any` е специален тип (универсален supertype), който може да се използва за представяне на стойност от произволен тип (примитиви, обекти, масиви, функции, грешки, символи). Често се използва в ситуации, когато типът на дадена стойност не е известен по време на компилация или при работа със стойности от външни API-та или библиотеки, които нямат TypeScript типизации.

Използвайки типа `any`, вие указвате на TypeScript компилатора, че стойностите трябва да бъдат представени без ограничения. За да максимизирате type безопасността в кода си, имайте предвид следното:

- Ограничете използването на `any` само до случаи, в които типът наистина не е известен.
- Не връщайте `any` тип от функция, тъй като това отслабва type безопасността на кода, който я използва.
- Вместо `any`, използвайте `@ts-ignore`, ако е необходимо да игнорирате предупреждение от компилатора.

```ts
let value: any;
value = true; // Валидно
value = 7; // Валидно
```

Unknown тип

В TypeScript, типът `unknown` представлява стойност с неизвестен тип. За разлика от типа `any`, който позволява всякакъв тип стойност, `unknown` изисква проверка на типа (type check) или type assertion, преди да може да бъде използван по конкретен начин, като не са позволени

операции върху unknown, без първо да бъде уточнен или стеснен до по-специфичен тип.

Типът unknown може да бъде присвоен само на тип any и на самия тип unknown, като представлява type-safe алтернатива на any.

```
let value: unknown;

let value1: unknown = value; // Валидно
let value2: any = value; // Валидно
let value3: boolean = value; // Невалидно
let value4: number = value; // Невалидно

const add = (a: unknown, b: unknown): number | undefined =>
    typeof a === 'number' && typeof b === 'number' ? a + b :
undefined;
console.log(add(1, 2)); // 3
console.log(add('x', 2)); // undefined
```

Void тип

Типът void се използва, за да покаже, че функция не връща стойност.

```
const sayHello = (): void => {
    console.log('Hello!');
};
```

Never тип

Типът never представлява стойности, които никога не се появяват. Той се използва за обозначаване на функции или изрази, които никога не връщат стойност или хвърлят грешка.

Например безкраен цикъл:

```
const infiniteLoop = (): never => {
    while (true) {
        // do something
    }
};
```

Хвърляне на грешка:

```
const throwError = (message: string): never => {
    throw new Error(message);
};
```

Типът `never` е полезен за гарантиране на type безопасност и откриване на потенциални грешки във вашия код. Той помага на TypeScript да анализира и извежда по-прецизни типове, когато се използва в комбинация с други типове и конструкции за control flow, например:

```
type Direction = 'up' | 'down';
const move = (direction: Direction): void => {
    switch (direction) {
        case 'up':
            // move up
            break;
        case 'down':
            // move down
            break;
        default:
            const exhaustiveCheck: never = direction;
                        throw new Error(`Unhandled direction:
${exhaustiveCheck}`);
    }
};
```

Interface и Type

Общ синтаксис

В TypeScript, interface дефинират структурата на обекти, като задават имената и типовете на свойствата или методите, които даден обект трябва да има. Общият синтаксис за дефиниране на interface в TypeScript е следният:

```
interface InterfaceName {
    property1: Type1;
    // ...
    method1(arg1: ArgType1, arg2: ArgType2): ReturnType;
    // ...
}
```

По подобен начин се дефинира и type:

```
type TypeName = {
    property1: Type1;
    // ...
    method1(arg1: ArgType1, arg2: ArgType2): ReturnType;
    // ...
};
```

`interface InterfaceName` или `type TypeName`: Дефинира името на interface. `property1: Type1`: Определя свойствата на interface заедно със съответните им типове. Могат да бъдат дефинирани множество свойства, разделени със точка и запетая. `method1(arg1: ArgType1, arg2: ArgType2): ReturnType;`: Определя методите на interface. Методите се дефинират чрез име, последвано от списък с параметри в скоби и тип на върнатата стойност. Могат да бъдат дефинирани множество методи, разделени със точка и запетая.

Пример за interface:

```
interface Person {
    name: string;
    age: number;
    greet(): void;
}
```

Пример за type:

```
type TypeName = {
    property1: string;
    method1(arg1: string, arg2: string): string;
};
```

В TypeScript, типовете се използват за дефиниране на структурата на данни и за прилагане на проверка на типовете (type checking). Съществуват различни често използвани синтаксиси за дефиниране на типове в TypeScript, в зависимост от конкретния случай на употреба. Ето някои примери:

Основни типове

```
let myNumber: number = 123; // number тип
let myBoolean: boolean = true; // boolean тип
let myArray: string[] = ['a', 'b']; // масив от string
let myTuple: [string, number] = ['a', 123]; // tuple
```

Обекти и Interfaces

```
const x: { name: string; age: number } = { name: 'Simon', age:
7 };
```

Union и Intersection типове

```
type MyType = string | number; // Union тип
let myUnion: MyType = 'hello'; // Може да бъде string
myUnion = 123; // Или number
```

```
type TypeA = { name: string };
type TypeB = { age: number };
type CombinedType = TypeA & TypeB; // Intersection тип
let myCombined: CombinedType = { name: 'John', age: 25 }; //
Обект със свойства name и age
```

Вградени примитивни типове

TypeScript има няколко вградени примитивни типа, които могат да се използват за дефиниране на променливи, параметри на функции и типове на върнатите стойности:

- `number`: Представя числови стойности, включително цели и числа с плаваща запетая.
- `string`: Представя текстови данни.
- `boolean`: Представя логически стойности, които могат да бъдат true или false.
- `null`: Представя липса на стойност.
- `undefined`: Представя стойност, която не е присвоена или не е дефинирана.
- `symbol`: Представя уникален идентификатор. Symbol обикновено се използва като ключ за свойства на обекти.
- `bigint`: Представя цели числа с произволна големина.
- `any`: Представя динамичен или неизвестен тип. Променливи от тип any могат да съдържат стойности от всякакъв тип и заобикалят проверката на типовете.
- `void`: Представя липса на тип. Обикновено се използва като тип на върната стойност на функции, които не връщат стойност.
- `never`: Представя тип за стойности, които никога не се появяват. Обикновено се използва като тип на върната

стойност на функции, които хвърлят грешка или влизат в безкраен цикъл.

Често използвани вградени JS обекти

TypeScript е надмножество (superset) на JavaScript и включва всички често използвани вградени JavaScript обекти. Можете да намерите обширен списък с тези обекти в документацията на Mozilla Developer Network (MDN): https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects

Ето списък с някои често използвани вградени JavaScript обекти:

- Function
- Object
- Boolean
- Error
- Number
- BigInt
- Math
- Date
- String
- RegExp
- Array
- Map
- Set
- Promise
- Intl

Overloads

Function overloads в TypeScript позволяват да дефинирате множество сигнатури на функция за едно и също име на функция, което позволява тя да бъде извиквана по различни начини. Ето пример:

```
// Overloads
function sayHi(name: string): string;
function sayHi(names: string[]): string[];

// Implementation
function sayHi(name: unknown): unknown {
    if (typeof name === 'string') {
        return `Hi, ${name}!`;
    } else if (Array.isArray(name)) {
        return name.map(name => `Hi, ${name}!`);
    }
    throw new Error('Invalid value');
}

sayHi('xx'); // Валидно
sayHi(['aa', 'bb']); // Валидно
```

Ето още един пример за използване на function overloads в рамките на `class`:

```
class Greeter {
    message: string;

    constructor(message: string) {
        this.message = message;
    }

    // overload
    sayHi(name: string): string;
    sayHi(names: string[]): ReadonlyArray<string>;

    // implementation
```

```
    sayHi(name: unknown): unknown {
        if (typeof name === 'string') {
            return `${this.message}, ${name}!`;
        } else if (Array.isArray(name)) {
                    return  name.map(name  =>  `${this.message},
${name}!`);
        }
        throw new Error('value is invalid');
    }
}
console.log(new Greeter('Hello').sayHi('Simon'));
```

Merging и Extension

Merging и extension се отнасят до две различни концепции, свързани с работа с типове и interfaces.

Merging позволява да комбинирате множество декларации със същото име в една дефиниция, например когато дефинирате interface със същото име повече от веднъж:

```
interface X {
    a: string;
}

interface X {
    b: number;
}

const person: X = {
    a: 'a',
    b: 7,
};
```

Extension се отнася до възможността да разширявате или наследявате съществуващи типове или interfaces, за да

създавате нови. Това е механизъм за добавяне на допълнителни свойства или методи към съществуващ тип, без да се променя оригиналната му дефиниция. Пример:

```
interface Animal {
    name: string;
    eat(): void;
}

interface Bird extends Animal {
    sing(): void;
}

const dog: Bird = {
    name: 'Bird 1',
    eat() {
        console.log('Eating');
    },
    sing() {
        console.log('Singing');
    },
};
```

Разлики между Type и Interface

Declaration merging (augmentation):

Interfaces поддържат declaration merging, което означава, че можете да дефинирате множество interfaces със същото име и TypeScript ще ги обедини в един interface с комбинирани свойства и методи. От друга страна, types не поддържат declaration merging. Това е полезно, когато искате да добавите допълнителна функционалност или да разширите съществуващи типове, без да променяте

оригиналните дефиниции или да поправяте липсващи или некоректни типове.

```
interface A {
    x: string;
}
interface A {
    y: string;
}
const j: A = {
    x: 'xx',
    y: 'yy',
};
```

Разширяване на други типове/interfaces:

Както types, така и interfaces могат да разширяват други типове/interfaces, но синтаксисът е различен. При interfaces използвате ключовата дума `extends`, за да наследите свойства и методи от други interfaces. Въпреки това, interface не може да разширява сложен тип като union тип.

```
interface A {
    x: string;
    y: number;
}
interface B extends A {
    z: string;
}
const car: B = {
    x: 'x',
    y: 123,
    z: 'z',
};
```

При types използвате оператора `&`, за да комбинирате множество типове в един тип (intersection).

```ts
interface A {
    x: string;
    y: number;
}

type B = A & {
    j: string;
};

const c: B = {
    x: 'x',
    y: 123,
    j: 'j',
};
```

Union и Intersection типове:

Types са по-гъвкави при дефиниране на Union и Intersection типове. С ключовата дума `type` можете лесно да създавате union типове чрез оператора `|` и intersection типове чрез оператора `&`. Докато interfaces могат индиректно да представят union типове, те нямат вградена поддръжка за intersection типове.

```ts
type Department = 'dep-x' | 'dep-y'; // Union

type Person = {
    name: string;
    age: number;
};

type Employee = {
    id: number;
    department: Department;
};

type EmployeeInfo = Person & Employee; // Intersection
```

Пример с interfaces:

```
interface A {
    x: 'x';
}
interface B {
    y: 'y';
}

type C = A | B; // Union от interfaces
```

Class

Общ синтаксис на Class

Ключовата дума `class` се използва в TypeScript за дефиниране на клас. По-долу е показан пример:

```
class Person {
    private name: string;
    private age: number;
    constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }
    public sayHi(): void {
        console.log(
                    `Hello, my name is ${this.name} and I am
${this.age} years old.`
        );
    }
}
```

Ключовата дума `class` се използва за дефиниране на клас с име “Person”.

Класът има две private свойства: name от тип `string` и age от тип `number`.

Конструкторът се дефинира чрез ключовата дума `constructor`. Той приема name и age като параметри и ги присвоява на съответните свойства.

Класът има `public` метод с име sayHi, който извежда поздравително съобщение.

За да създадете инстанция на клас в TypeScript, можете да използвате ключовата дума `new`, последвана от името на класа и скоби `()`. Например:

```
const myObject = new Person('John Doe', 25);
myObject.sayHi(); // Output: Hello, my name is John Doe and I
am 25 years old.
```

Constructor

Конструкторите са специални методи в рамките на class, които се използват за инициализиране на свойствата на обекта при създаване на инстанция на класа.

```
class Person {
    public name: string;
    public age: number;

    constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }

    sayHello() {
        console.log(
```

```
            `Hello, my name is ${this.name} and I'm ${this.age}
years old.`
        );
    }
}

const john = new Person('Simon', 17);
john.sayHello();
```

Възможно е да се използва overload на constructor със следния синтаксис:

```
type Sex = 'm' | 'f';

class Person {
    name: string;
    age: number;
    sex: Sex;

    constructor(name: string, age: number, sex?: Sex);
    constructor(name: string, age: number, sex: Sex) {
        this.name = name;
        this.age = age;
        this.sex = sex ?? 'm';
    }
}

const p1 = new Person('Simon', 17);
const p2 = new Person('Alice', 22, 'f');
```

В TypeScript е възможно да се дефинират множество constructor overload-и, но може да има само една имплементация, която трябва да бъде съвместима с всички overload-и. Това може да се постигне чрез използване на опционални параметри.

```
class Person {
    name: string;
    age: number;

    constructor();
    constructor(name: string);
    constructor(name: string, age: number);
    constructor(name?: string, age?: number) {
        this.name = name ?? 'Unknown';
        this.age = age ?? 0;
    }

    displayInfo() {
        console.log(`Name: ${this.name}, Age: ${this.age}`);
    }
}

const person1 = new Person();
person1.displayInfo(); // Name: Unknown, Age: 0

const person2 = new Person('John');
person2.displayInfo(); // Name: John, Age: 0

const person3 = new Person('Jane', 25);
person3.displayInfo(); // Name: Jane, Age: 25
```

Private и Protected конструктори

В TypeScript конструкторите могат да бъдат маркирани като private или protected, което ограничава тяхната достъпност и използване.

Private конструктори: Могат да бъдат извиквани само в рамките на самия class. Private конструкторите често се използват, когато искате да наложите singleton pattern или

да ограничите създаването на инстанции чрез factory метод в рамките на класа.

Protected конструктори: Protected конструкторите са полезни, когато искате да създадете базов клас, който не трябва да бъде инстанциран директно, но може да бъде разширяван от наследяващи класове.

```
class BaseClass {
    protected constructor() {}
}

class DerivedClass extends BaseClass {
    private value: number;

    constructor(value: number) {
        super();
        this.value = value;
    }
}

// Опит за директно създаване на инстанция на базовия клас ще доведе до грешка
// const baseObj = new BaseClass(); // Error: Constructor of class 'BaseClass' is protected.

// Създаване на инстанция на наследения клас
const derivedObj = new DerivedClass(10);
```

Модификатори за достъп

Модификаторите за достъп `private`, `protected` и `public` се използват за контролиране на видимостта и достъпността на членовете на клас (като свойства и методи) в TypeScript. Тези модификатори са от съществено значение за прилагане на енкапсулация и за определяне на граници

при достъп и промяна на вътрешното състояние на един клас.

Модификаторът `private` ограничава достъпа до члена на класа само в рамките на самия клас.

Модификаторът `protected` позволява достъп до члена на класа в рамките на самия клас и неговите наследници.

Модификаторът `public` предоставя неограничен достъп до члена на класа, като позволява той да бъде достъпен отвсякъде.

Get и Set

Getter-и и setter-и са специални методи, които позволяват да дефинирате персонализирано поведение при достъп и промяна на свойства на клас. Те позволяват да капсулирате вътрешното състояние на обект и да добавите допълнителна логика при вземане (get) или задаване (set) на стойности. В TypeScript getter-ите и setter-ите се дефинират съответно чрез ключовите думи `get` и `set`. Ето пример:

```
class MyClass {
    private _myProperty: string;

    constructor(value: string) {
        this._myProperty = value;
    }
    get myProperty(): string {
        return this._myProperty;
    }
    set myProperty(value: string) {
        this._myProperty = value;
```

```
    }
}
```

Auto-accessors в класове

TypeScript версия 4.9 добавя поддръжка за auto-accessors — предстояща функционалност в ECMAScript. Те наподобяват свойства на клас, но се декларират с ключовата дума `accessor`.

```
class Animal {
    accessor name: string;

    constructor(name: string) {
        this.name = name;
    }
}
```

Auto-accessors се "десъгарират" (de-sugared) до private `get` и `set` аксесори, които работят върху недостъпно свойство.

```
class Animal {
    #__name: string;

    get name() {
        return this.#__name;
    }
    set name(value: string) {
        this.#__name = value;
    }

    constructor(name: string) {
        this.name = name;
    }
}
```

this

В TypeScript, ключовата дума `this` се отнася до текущата инстанция на класа в рамките на неговите методи или конструктори. Тя позволява достъп и промяна на свойствата и методите на класа от неговия собствен обхват. Тя предоставя начин за достъп и манипулиране на вътрешното състояние на обект в рамките на неговите методи.

```
class Person {
    private name: string;
    constructor(name: string) {
        this.name = name;
    }
    public introduce(): void {
        console.log(`Hello, my name is ${this.name}.`);
    }
}

const person1 = new Person('Alice');
person1.introduce(); // Hello, my name is Alice.
```

Parameter Properties

Parameter properties позволяват да декларирате и инициализирате свойства на клас директно в параметрите на конструктора, като избягвате излишен boilerplate код. Пример:

```
class Person {
    constructor(
        private name: string,
        public age: number
    ) {
        // Ключовите думи "private" и "public" в конструктора
          // автоматично декларират и инициализират съответните
свойства на класа.
```

```
    }
    public introduce(): void {
        console.log(
                    `Hello, my name is ${this.name} and I am
${this.age} years old.`
        );
    }
}
const person = new Person('Alice', 25);
person.introduce();
```

Абстрактни класове

Абстрактните класове се използват в TypeScript основно за наследяване. Те предоставят начин за дефиниране на общи свойства и методи, които могат да бъдат наследявани от под-класове. Това е полезно, когато искате да дефинирате общо поведение и да наложите под-класовете да имплементират определени методи. Те предоставят начин за създаване на йерархия от класове, където абстрактният базов клас осигурява общ интерфейс и функционалност за под-класовете.

```
abstract class Animal {
    protected name: string;

    constructor(name: string) {
        this.name = name;
    }

    abstract makeSound(): void;
}

class Cat extends Animal {
    makeSound(): void {
        console.log(`${this.name} meows.`);
```

```
    }
}

const cat = new Cat('Whiskers');
cat.makeSound(); // Output: Whiskers meows.
```

C generics

Класовете с generics позволяват да дефинирате преизползваеми класове, които могат да работят с различни типове.

```
class Container<T> {
    private item: T;

    constructor(item: T) {
        this.item = item;
    }

    getItem(): T {
        return this.item;
    }

    setItem(item: T): void {
        this.item = item;
    }
}

const container1 = new Container<number>(42);
console.log(container1.getItem()); //  42

const container2 = new Container<string>('Hello');
container2.setItem('World');
console.log(container2.getItem()); // World
```

Decorators

Decorators предоставят механизъм за добавяне на metadata, промяна на поведение, валидиране или разширяване на функционалността на даден елемент. Те са функции, които се изпълняват по време на runtime. Могат да бъдат прилагани множество decorators върху една декларация.

Decorators са експериментална функционалност и следващите примери са съвместими само с TypeScript версия 5 или по-нова, използвайки ES6.

За версии на TypeScript преди 5, те трябва да бъдат активирани чрез свойството `experimentalDecorators` във вашия `tsconfig.json` или чрез използване на `--experimentalDecorators` в командния ред (но следният пример няма да работи).

Някои от често срещаните случаи на употреба на decorators включват:

- Следене на промени в свойства.
- Следене на извиквания на методи.
- Добавяне на допълнителни свойства или методи.
- Runtime валидиране.
- Автоматична сериализация и десериализация.
- Логване.
- Авторизация и автентикация.
- Защита от грешки.

Забележка: Decorators във версия 5 не позволяват декориране на параметри.

Видове decorators:

Class Decorators

Class Decorators са полезни за разширяване на съществуващ клас, например чрез добавяне на свойства или методи, или събиране на инстанции на даден клас. В следния пример добавяме метод `toString`, който преобразува класа в текстово представяне.

```
type Constructor<T = {}> = new (...args: any[]) => T;

function toString<Class extends Constructor>(
    Value: Class,
    context: ClassDecoratorContext<Class>
) {
    return class extends Value {
        constructor(...args: any[]) {
            super(...args);
            console.log(JSON.stringify(this));
            console.log(JSON.stringify(context));
        }
    };
}

@toString
class Person {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    greet() {
        return 'Hello, ' + this.name;
    }
}
const person = new Person('Simon');
/* Logs:
```

```
{"name":"Simon"}
{"kind":"class","name":"Person"}
*/
```

Property Decorator

Property decorators са полезни за промяна на поведението на дадено свойство, например чрез промяна на началните му стойности. В следния код имаме скрипт, който задава стойността на свойство винаги да бъде с главни букви:

```
function upperCase<T>(
    target: undefined,
    context: ClassFieldDecoratorContext<T, string>
) {
    return function (this: T, value: string) {
        return value.toUpperCase();
    };
}

class MyClass {
    @upperCase
    prop1 = 'hello!';
}

console.log(new MyClass().prop1); // Logs: HELLO!
```

Method Decorator

Method decorators позволяват да променяте или разширявате поведението на методи. По-долу е даден пример за прост logger:

```
function log<This, Args extends any[], Return>(
    target: (this: This, ...args: Args) => Return,
    context: ClassMethodDecoratorContext<
```

```
        This,
        (this: This, ...args: Args) => Return
    >
) {
    const methodName = String(context.name);

    function replacementMethod(this: This, ...args: Args):
Return {
        console.log(`LOG: Entering method '${methodName}'.`);
        const result = target.call(this, ...args);
        console.log(`LOG: Exiting method '${methodName}'.`);
        return result;
    }

    return replacementMethod;
}

class MyClass {
    @log
    sayHello() {
        console.log('Hello!');
    }
}

new MyClass().sayHello();
```

Логва:

```
LOG: Entering method 'sayHello'.
Hello!
LOG: Exiting method 'sayHello'.
```

Декоратори за Getter и Setter

Декораторите за getter и setter позволяват да променяте или разширявате поведението на аксесорите на клас. Те са

полезни например за валидиране на присвоявания на свойства. Ето един прост пример за getter декоратор:

```
function range<This, Return extends number>(min: number, max:
number) {
    return function (
        target: (this: This) => Return,
        context: ClassGetterDecoratorContext<This, Return>
    ) {
        return function (this: This): Return {
            const value = target.call(this);
            if (value < min || value > max) {
                throw 'Невалидно';
            }
            Object.defineProperty(this, context.name, {
                value,
                enumerable: true,
            });
            return value;
        };
    };
}

class MyClass {
    private _value = 0;

    constructor(value: number) {
        this._value = value;
    }
    @range(1, 100)
    get getValue(): number {
        return this._value;
    }
}

const obj = new MyClass(10);
console.log(obj.getValue); // Валидно: 10
```

```
const obj2 = new MyClass(999);
console.log(obj2.getValue); // Throw: Невалидно!
```

Metadata за декоратори

Decorator Metadata улеснява процеса за decorators да прилагат и използват metadata във всеки клас. Те могат да достъпват ново metadata свойство върху context обекта, което може да служи като ключ както за примитиви, така и за обекти. Metadata информацията може да бъде достъпена от класа чрез `Symbol.metadata`.

Metadata може да се използва за различни цели, като дебъгване, сериализация или dependency injection с decorators.

```
//@ts-ignore
Symbol.metadata ??= Symbol('Symbol.metadata'); // Прост polyfill

type Context =
    | ClassFieldDecoratorContext
    | ClassAccessorDecoratorContext
     | ClassMethodDecoratorContext; // Context съдържа metadata за свойства: DecoratorMetadata

function setMetadata(_target: any, context: Context) {
    // Задава metadata обект с примитивна стойност
    context.metadata[context.name] = true;
}

class MyClass {
    @setMetadata
    a = 123;

    @setMetadata
```

```
    accessor b = 'b';

    @setMetadata
    fn() {}
}

const metadata = MyClass[Symbol.metadata]; // Взима metadata информация

console.log(JSON.stringify(metadata)); // {"bar":true,"baz":true,"foo":true}
```

Наследяване

Наследяването се отнася до механизма, при който един клас може да наследява свойства и методи от друг клас, наречен базов клас или superclass. Производният клас, наричан още child class или subclass, може да разширява и специализира функционалността на базовия клас чрез добавяне на нови свойства и методи или чрез презаписване (override) на съществуващи.

```
class Animal {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    speak(): void {
        console.log('The animal makes a sound');
    }
}

class Dog extends Animal {
    breed: string;
```

```
    constructor(name: string, breed: string) {
        super(name);
        this.breed = breed;
    }

    speak(): void {
        console.log('Woof! Woof!');
    }
}

// Създаване на инстанция на базовия клас
const animal = new Animal('Generic Animal');
animal.speak(); // The animal makes a sound

// Създаване на инстанция на производния клас
const dog = new Dog('Max', 'Labrador');
dog.speak(); // Woof! Woof!"
```

TypeScript не поддържа множествено наследяване в традиционния смисъл и вместо това позволява наследяване от един базов клас. TypeScript поддържа множество interfaces. Един interface може да дефинира договор за структурата на обект, а един клас може да имплементира множество interfaces. Това позволява на класа да наследява поведение и структура от множество източници.

```
interface Flyable {
    fly(): void;
}

interface Swimmable {
    swim(): void;
}

class FlyingFish implements Flyable, Swimmable {
    fly() {
```

```
        console.log('Flying...');
    }

    swim() {
        console.log('Swimming...');
    }
}

const flyingFish = new FlyingFish();
flyingFish.fly();
flyingFish.swim();
```

Ключовата дума `class` в TypeScript, подобно на JavaScript, често се нарича syntactic sugar. Тя е въведена в ECMAScript 2015 (ES6), за да предостави по-познат синтаксис за създаване и работа с обекти по клас-ориентиран начин. Въпреки това е важно да се отбележи, че TypeScript, като надмножество на JavaScript, в крайна сметка се компилира до JavaScript, който остава прототипно базиран в основата си.

Статични членове

TypeScript поддържа статични членове. За достъп до статичните членове на клас можете да използвате името на класа, последвано от точка, без да е необходимо да създавате инстанция.

```
class OfficeWorker {
    static memberCount: number = 0;

    constructor(private name: string) {
        OfficeWorker.memberCount++;
    }
}
```

```
const w1 = new OfficeWorker('James');
const w2 = new OfficeWorker('Simon');
const total = OfficeWorker.memberCount;
console.log(total); // 2
```

Инициализация на свойства

Съществуват няколко начина за инициализиране на свойства на клас в TypeScript:

Inline:

В следния пример тези начални стойности ще бъдат използвани при създаване на инстанция на класа.

```
class MyClass {
    property1: string = 'default value';
    property2: number = 42;
}
```

В конструктора:

```
class MyClass {
    property1: string;
    property2: number;

    constructor() {
        this.property1 = 'default value';
        this.property2 = 42;
    }
}
```

Чрез параметри на конструктора:

```
class MyClass {
    constructor(
        private property1: string = 'default value',
```

```
        public property2: number = 42
    ) {
        // Няма нужда да присвояваме стойностите на свойствата
изрично.
    }
    log() {
        console.log(this.property2);
    }
}
const x = new MyClass();
x.log();
```

Method overloading

Method overloading позволява на един клас да има множество методи със същото име, но с различни типове параметри или различен брой параметри. Това позволява методът да бъде извикван по различни начини в зависимост от подадените аргументи.

```
class MyClass {
    add(a: number, b: number): number; // Overload signature 1
    add(a: string, b: string): string; // Overload signature 2

      add(a: number | string, b: number | string): number |
string {
        if (typeof a === 'number' && typeof b === 'number') {
            return a + b;
        }
        if (typeof a === 'string' && typeof b === 'string') {
            return a.concat(b);
        }
        throw new Error('Invalid arguments');
    }
}

const r = new MyClass();
console.log(r.add(10, 5)); // Logs 15
```

Generics

Generics позволяват да създавате преизползваеми компоненти и функции, които могат да работят с множество типове. С generics можете да параметризирате типове, функции и interfaces, позволявайки им да работят с различни типове без да ги указвате изрично предварително.

Generics правят кода по-гъвкав и преизползваем.

Generic тип

За да дефинирате generic тип, използвате ъглови скоби (<>), в които посочвате type параметрите, например:

```
function identity<T>(arg: T): T {
    return arg;
}
const a = identity('x');
const b = identity(123);

const getLen = <T,>(data: ReadonlyArray<T>) => data.length;
const len = getLen([1, 2, 3]);
```

Generic класове

Generics могат да се прилагат и към класове, като по този начин те могат да работят с множество типове чрез използване на type параметри. Това е полезно за създаване на преизползваеми класове, които работят с различни типове данни, като същевременно запазват type безопасността.

```
class Container<T> {
    private item: T;

    constructor(item: T) {
        this.item = item;
    }

    getItem(): T {
        return this.item;
    }
}

const numberContainer = new Container<number>(123);
console.log(numberContainer.getItem()); // 123

const stringContainer = new Container<string>('hello');
console.log(stringContainer.getItem()); // hello
```

Ограничения при generics

Generic параметрите могат да бъдат ограничени чрез ключовата дума `extends`, последвана от тип или interface, който type параметърът трябва да удовлетворява.

В следния пример T трябва да съдържа свойство `length`, за да бъде валиден:

```
const printLen = <T extends { length: number }>(value: T): void
=> {
    console.log(value.length);
};

printLen('Hello'); // 5
printLen([1, 2, 3]); // 3
printLen({ length: 10 }); // 10
printLen(123); // Невалидно
```

Интересна функционалност на generics, въведена във версия 3.4 RC, е Higher order function type inference, която въвежда пропагирани generic type аргументи:

```
declare function pipe<A extends any[], B, C>(
    ab: (...args: A) => B,
    bc: (b: B) => C
): (...args: A) => C;

declare function list<T>(a: T): T[];
declare function box<V>(x: V): { value: V };

const listBox = pipe(list, box); // <T>(a: T) => { value: T[] }
const boxList = pipe(box, list); // <V>(x: V) => { value: V }[]
```

Тази функционалност позволява по-лесно type-safe pointfree стил на програмиране, който е често срещан във функционалното програмиране.

Контекстуално стесняване при generics

Контекстуалното стесняване при generics е механизъм в TypeScript, който позволява на компилатора да стеснява типа на generic параметър въз основа на контекста, в който се използва. Това е полезно при работа с generic типове в условни конструкции:

```
function process<T>(value: T): void {
    if (typeof value === 'string') {
        // Стесняване до тип 'string'
        console.log(value.length);
    } else if (typeof value === 'number') {
        // Стесняване до тип 'number'
        console.log(value.toFixed(2));
    }
}
```

```
process('hello'); // 5
process(3.14159); // 3.14
```

Изтрити структурни типове

В TypeScript обектите не е необходимо да съвпадат с конкретен, точен тип. Например, ако създадем обект, който покрива изискванията на даден interface, можем да използваме този обект на места, където този interface се изисква, дори да няма изрична връзка между тях. Пример:

```
type NameProp1 = {
    prop1: string;
};

function log(x: NameProp1) {
    console.log(x.prop1);
}

const obj = {
    prop2: 123,
    prop1: 'Origin',
};

log(obj); // Валидно
```

Namespacing

В TypeScript namespaces се използват за организиране на кода в логически контейнери, предотвратяване на конфликти в имената и групиране на свързан код. Използването на ключовата дума `export` позволява достъп до namespace от външни модули.

```
export namespace MyNamespace {
    export interface MyInterface1 {
        prop1: boolean;
    }
    export interface MyInterface2 {
        prop2: string;
    }
}

const a: MyNamespace.MyInterface1 = {
    prop1: true,
};
```

Symbols

Symbols са примитивен тип данни, който представлява неизменима стойност, гарантирано уникална в рамките на изпълнението на програмата.

Symbols могат да се използват като ключове за свойства на обекти и предоставят начин за създаване на не-енумерируеми свойства.

```
const key1: symbol = Symbol('key1');
const key2: symbol = Symbol('key2');

const obj = {
    [key1]: 'value 1',
    [key2]: 'value 2',
};

console.log(obj[key1]); // value 1
console.log(obj[key2]); // value 2
```

В WeakMap и WeakSet, symbols вече могат да се използват като ключове.

Triple-Slash директиви

Triple-slash директивите са специални коментари, които предоставят инструкции към компилатора как да обработи даден файл. Тези директиви започват с три наклонени черти (`///`) и обикновено се поставят в началото на TypeScript файл, като нямат ефект върху runtime поведението.

Triple-slash директивите се използват за реферирane на външни зависимости, указване на начина на зареждане на модули, активиране/деактивиране на определени компилаторни функции и други. Няколко примера:

Реферирane на декларационен файл:

```
/// <reference path="path/to/declaration/file.d.ts" />
```

Указване на формат на модул:

```
/// <amd|commonjs|system|umd|es6|es2015|none>
```

Активиране на компилаторни опции, например strict режим:

```
/// <strict|noImplicitAny|noUnusedLocals|noUnusedParameters>
```

Манипулация на типове

Създаване на типове от типове

Възможно е да се създават нови типове чрез комбиниране, манипулиране или трансформиране на съществуващи типове.

Intersection типове (&):

Позволяват комбиниране на множество типове в един:

```
type A = { foo: number };
type B = { bar: string };
type C = A & B; // Intersection of A and B
const obj: C = { foo: 42, bar: 'hello' };
```

Union типове (|):

Позволяват дефиниране на тип, който може да бъде един от няколко:

```
type Result = string | number;
const value1: Result = 'hello';
const value2: Result = 42;
```

Mapped типове:

Позволяват трансформиране на свойствата на съществуващ тип за създаване на нов тип:

```
type Mutable<T> = {
    readonly [P in keyof T]: T[P];
};
type Person = {
    name: string;
    age: number;
};
type ImmutablePerson = Mutable<Person>; // Свойствата стават read-only
```

Условни типове:

Позволяват създаване на типове въз основа на условия:

```
type ExtractParam<T> = T extends (param: infer P) => any ? P :
never;
type MyFunction = (name: string) => number;
type ParamType = ExtractParam<MyFunction>; // string
```

Indexed Access Types

В TypeScript е възможно да се достъпват и манипулират типовете на свойства в друг тип чрез индекс, `Type[Key]`.

```
type Person = {
    name: string;
    age: number;
};

type AgeType = Person['age']; // number

type MyTuple = [string, number, boolean];
type MyType = MyTuple[2]; // boolean
```

Utility типове

Съществуват няколко вградени utility типа, които могат да се използват за манипулиране на типове. По-долу е списък с най-често използваните:

Awaited<T>

Създава тип, който рекурсивно “разопакова” Promise типове.

```
type A = Awaited<Promise<string>>; // string
```

Partial<T>

Създава тип, в който всички свойства на T са опционални.

```
type Person = {
    name: string;
    age: number;
};

type A = Partial<Person>; // { name?: string | undefined; age?:
number | undefined; }
```

Required<T>

Създава тип, в който всички свойства на T са задължителни.

```
type Person = {
    name?: string;
    age?: number;
};

type A = Required<Person>; // { name: string; age: number; }
```

Readonly<T>

Създава тип, в който всички свойства на T са readonly.

```
type Person = {
    name: string;
    age: number;
};

type A = Readonly<Person>;

const a: A = { name: 'Simon', age: 17 };
a.name = 'John'; // Невалидно
```

Record<K, T>

Създава тип с набор от свойства K от тип T.

```
type Product = {
    name: string;
    price: number;
};

const products: Record<string, Product> = {
    apple: { name: 'Apple', price: 0.5 },
    banana: { name: 'Banana', price: 0.25 },
};

console.log(products.apple); // { name: 'Apple', price: 0.5 }
```

Pick<T, K>

Създава тип чрез избиране на конкретни свойства K от T.

```
type Product = {
    name: string;
    price: number;
};

type Price = Pick<Product, 'price'>; // { price: number; }
```

Omit<T, K>

Създава тип чрез премахване на конкретни свойства K от T.

```
type Product = {
    name: string;
    price: number;
};
```

```
type Name = Omit<Product, 'price'>; // { name: string; }
```

Exclude<T, U>

Създава тип чрез изключване на всички стойности от тип U от T.

```
type Union = 'a' | 'b' | 'c';
type MyType = Exclude<Union, 'a' | 'c'>; // b
```

Extract<T, U>

Създава тип чрез извличане на всички стойности от тип U от T.

```
type Union = 'a' | 'b' | 'c';
type MyType = Extract<Union, 'a' | 'c'>; // a | c
```

NonNullable<T>

Създава тип чрез премахване на null и undefined от T.

```
type Union = 'a' | null | undefined | 'b';
type MyType = NonNullable<Union>; // 'a' | 'b'
```

Parameters<T>

Извлича типовете на параметрите на функция T.

```
type Func = (a: string, b: number) => void;
type MyType = Parameters<Func>; // [a: string, b: number]
```

ConstructorParameters<T>

Извлича типовете на параметрите на конструктор T.

```
class Person {
    constructor(
        public name: string,
        public age: number
    ) {}
}
type PersonConstructorParams = ConstructorParameters<typeof
Person>; // [name: string, age: number]
const params: PersonConstructorParams = ['John', 30];
const person = new Person(...params);
console.log(person); // Person { name: 'John', age: 30 }
```

ReturnType<T>

Извлича типа на стойността, върната от функция T.

```
type Func = (name: string) => number;
type MyType = ReturnType<Func>; // number
```

InstanceType<T>

Извлича типа на инстанцията на клас T.

```
class Person {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    sayHello() {
        console.log(`Hello, my name is ${this.name}!`);
    }
}
```

```
type PersonInstance = InstanceType<typeof Person>;

const person: PersonInstance = new Person('John');

person.sayHello(); // Hello, my name is John!
```

ThisParameterType<T>

Извлича типа на `this` параметъра от функция T.

```
interface Person {
    name: string;
    greet(this: Person): void;
}
type  PersonThisType  =  ThisParameterType<Person['greet']>;  //
Person
```

OmitThisParameter<T>

Премахва `this` параметъра от функция T.

```
function capitalize(this: String) {
                        return      this[0].toUpperCase()      +
this.substring(1).toLowerCase();
}

type CapitalizeType = OmitThisParameter<typeof capitalize>; //
() => string
```

ThisType<T>

Служи като маркер за контекстуален `this` тип.

```
type Logger = {
    log: (error: string) => void;
};
```

```
let helperFunctions: { [name: string]: Function } &
ThisType<Logger> = {
    hello: function () {
         this.log('some error'); // Валидно as "log" is a part
of "this".
        this.update(); // Невалидно
    },
};
```

Uppercase<T>

Прави текста на входния тип T с главни букви.

```
type MyType = Uppercase<'abc'>; // "ABC"
```

Lowercase<T>

Прави текста на входния тип T с малки букви.

```
type MyType = Lowercase<'ABC'>; // "abc"
```

Capitalize<T>

Прави първата буква на входния тип T главна.

```
type MyType = Capitalize<'abc'>; // "Abc"
```

Uncapitalize<T>

Прави първата буква на входния тип T малка.

```
type MyType = Uncapitalize<'Abc'>; // "abc"
```

NoInfer<T>

NoInfer е utility тип, създаден да блокира автоматичното извеждане (inference) на типове в рамките на generic функция.

Пример:

```
// Автоматично извеждане на типове в generic функция
function fn<T extends string>(x: T[], y: T) {
    return x.concat(y);
}
const r = fn(['a', 'b'], 'c'); // Type here is ("a" | "b" |
"c")[]
```

С NoInfer:

```
// Примерна функция, която използва NoInfer, за да предотврати
type inference
function fn2<T extends string>(x: T[], y: NoInfer<T>) {
    return x.concat(y);
}

const r2 = fn2(['a', 'b'], 'c'); // Error: Type Argument of
type '"c"' is not assignable to parameter of type '"a" | "b"'.
```

Други

Грешки и обработка на изключения

TypeScript позволява да прихващате и обработвате грешки, използвайки стандартните механизми на JavaScript:

Try-Catch-Finally блокове:

```
try {
    // Код, който може да хвърли грешка
} catch (error) {
    // Обработка на грешката
```

```
} finally {
    // Код, който винаги се изпълнява, finally е по избор
}
```

Можете също така да обработвате различни типове грешки:

```
try {
    // Код, който може да хвърли различни типове грешки
} catch (error) {
    if (error instanceof TypeError) {
        // Обработка на TypeError
    } else if (error instanceof RangeError) {
        // Обработка на RangeError
    } else {
        // Обработка на други грешки
    }
}
```

Потребителски типове грешки:

Възможно е да дефинирате по-специфични грешки чрез разширяване на класа Error:

```
class CustomError extends Error {
    constructor(message: string) {
        super(message);
        this.name = 'CustomError';
    }
}

throw new CustomError('This is a custom error.');
```

Mixin класове

Mixin класовете позволяват комбиниране и композиция на поведение от множество класове в един клас. Те

предоставят начин за преизползване и разширяване на функционалност без нужда от дълбоки вериги на наследяване.

```
abstract class Identifiable {
    name: string = '';
    logId() {
        console.log('id:', this.name);
    }
}
abstract class Selectable {
    selected: boolean = false;
    select() {
        this.selected = true;
        console.log('Select');
    }
    deselect() {
        this.selected = false;
        console.log('Deselect');
    }
}
class MyClass {
    constructor() {}
}

// Разширяване на MyClass с поведението на Identifiable и
Selectable
interface MyClass extends Identifiable, Selectable {}

// Функция за прилагане на mixins към клас
function applyMixins(source: any, baseCtors: any[]) {
    baseCtors.forEach(baseCtor => {

Object.getOwnPropertyNames(baseCtor.prototype).forEach(name   =>
{
            let descriptor = Object.getOwnPropertyDescriptor(
                baseCtor.prototype,
                name
```

```
            );
            if (descriptor) {
                  Object.defineProperty(source.prototype, name,
descriptor);
            }
        });
    });
}

// Прилагане на mixins към MyClass
applyMixins(MyClass, [Identifiable, Selectable]);
let o = new MyClass();
o.name = 'abc';
o.logId();
o.select();
```

Асинхронни функционалности

Тъй като TypeScript е надмножество на JavaScript, той поддържа вградените асинхронни функционалности на JavaScript, като:

Promises:

Promises са начин за обработка на асинхронни операции и техните резултати чрез методи като `.then()` и `.catch()` за обработка на успешни и неуспешни състояния.

За повече информация: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise

Async/await:

Ключовите думи async/await предоставят по-синхронно изглеждащ синтаксис за работа с Promises. Ключовата дума `async` се използва за дефиниране на асинхронна функция, а

`await` се използва в рамките на такава функция, за да спре изпълнението, докато Promise бъде изпълнен или отхвърлен.

За повече информация: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/await

Следните API-та са добре поддържани в TypeScript:

Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

Web Workers: https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API

Shared Workers: https://developer.mozilla.org/en-US/docs/Web/API/SharedWorker

WebSocket: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

Итератори и генератори

Както итераторите, така и генераторите са добре поддържани в TypeScript.

Итераторите са обекти, които имплементират iterator протокола, предоставяйки начин за достъп до елементите на колекция или последователност един по един. Те представляват структура, която съдържа указател към следващия елемент в итерацията. Имат метод `next()`, който

връща следващата стойност в последователността заедно с boolean стойност, указваща дали последователността е done.

```
class NumberIterator implements Iterable<number> {
    private current: number;

    constructor(
        private start: number,
        private end: number
    ) {
        this.current = start;
    }

    public next(): IteratorResult<number> {
        if (this.current <= this.end) {
            const value = this.current;
            this.current++;
            return { value, done: false };
        } else {
            return { value: undefined, done: true };
        }
    }

    [Symbol.iterator](): Iterator<number> {
        return this;
    }
}

const iterator = new NumberIterator(1, 3);

for (const num of iterator) {
    console.log(num);
}
```

Генераторите са специални функции, дефинирани със синтаксиса `function*`, които улесняват създаването на итератори. Те използват ключовата дума `yield`, за да

дефинират последователността от стойности и автоматично спират и възобновяват изпълнението, когато се изискват стойности.

Генераторите улесняват създаването на итератори и са особено полезни при работа с големи или безкрайни последователности.

Пример:

```
function* numberGenerator(start: number, end: number): Generator<number> {
    for (let i = start; i <= end; i++) {
        yield i;
    }
}

const generator = numberGenerator(1, 5);

for (const num of generator) {
    console.log(num);
}
```

TypeScript също така поддържа async итератори и async генератори.

За повече информация:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Generator

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator

TsDocs JSDoc Reference

Когато работите с JavaScript кодова база, е възможно да помогнете на TypeScript да извлече правилния тип, използвайки JSDoc коментари с допълнителни анотации за предоставяне на типова информация.

Пример:

```
/**
 * Изчислява степента на дадено число
 * @constructor
 * @param {number} base - Основната стойност на израза
 * @param {number} exponent - Степента на израза
 */
function power(base: number, exponent: number) {
    return Math.pow(base, exponent);
}
power(10, 2); // function power(base: number, exponent: number): number
```

Full documentation is provided to this link: https://www.typescriptlang.org/docs/handbook/jsdoc-supported-types.html

От версия 3.7 е възможно да се генерират `.d.ts` type definitions от JavaScript JSDoc синтаксис. Повече информация може да бъде намерена тук: https://www.typescriptlang.org/docs/handbook/declaration-files/dts-from-js.html

@types

Пакетите под организацията @types са специална naming convention, използвана за предоставяне на type definitions за съществуващи JavaScript библиотеки или модули. Например:

```
npm install --save-dev @types/lodash
```

ще инсталира type definitions за `lodash` във вашия проект.

За да допринесете към type definitions на @types пакет, изпратете pull request към: https://github.com/DefinitelyTyped/DefinitelyTyped.

JSX

JSX (JavaScript XML) е разширение на синтаксиса на JavaScript, което позволява да пишете HTML-подобен код директно в JavaScript или TypeScript файлове. Често се използва в React за дефиниране на HTML структура.

TypeScript разширява възможностите на JSX чрез type checking и static analysis.

За да използвате JSX, трябва да зададете `jsx` опцията в `tsconfig.json`. Две често използвани конфигурации:

- “preserve”: генерира .jsx файлове без промяна на JSX. TypeScript запазва JSX синтаксиса и не го трансформира. Подходящо, ако използвате инструмент като Babel.
- “react”: активира вградената JSX трансформация в TypeScript. Използва се React.createElement.

Всички опции: https://www.typescriptlang.org/tsconfig#jsx

ES6 Modules

TypeScript поддържа ES6 (ECMAScript 2015) и следващи версии. Това означава, че можете да използвате arrow

functions, template literals, класове, модули, destructuring и други.

За да активирате ES6 функционалности, задайте `target` в tsconfig.json.

Пример:

```
{
  "compilerOptions": {
    "target": "es6",
    "module": "es6",
    "moduleResolution": "node",
    "sourceMap": true,
    "outDir": "dist"
  },
  "include": ["src"]
}
```

ES7 Exponentiation Operator

Операторът за повдигане в степен (**) изчислява стойността, получена при повдигането на първия операнд в степен на втория операнд. Той работи по подобен начин като `Math.pow()`, но с допълнителната възможност да приема BigInts като операнди. TypeScript поддържа напълно този оператор, ако използвате `target` в файла tsconfig.json и сте задали `es2016` или по-нова версия.

```
console.log(2 ** (2 ** 2)); // 16
```

The for-await-of Statement

Това е JavaScript функционалност, напълно поддържана в TypeScript, която позволява итериране върху асинхронни

iterable обекти (от es2018).

```
async function* asyncNumbers(): AsyncIterableIterator<number> {
    yield Promise.resolve(1);
    yield Promise.resolve(2);
    yield Promise.resolve(3);
}

(async () => {
    for await (const num of asyncNumbers()) {
        console.log(num);
    }
})();
```

New target meta-property

В TypeScript можете да използвате метасвойството `new.target`, което ви позволява да установите дали дадена функция или конструктор е бил извикан чрез оператора `new`. То ви позволява да установите дали даден обект е бил създаден в резултат на извикване на конструктор.

```
class Parent {
    constructor() {
            console.log(new.target);  // Logs the constructor
function used to create an instance
    }
}

class Child extends Parent {
    constructor() {
        super();
    }
}

const parentX = new Parent(); // [Function: Parent]
const child = new Child(); // [Function: Child]
```

Dynamic Import Expressions

Възможно е модулите да се зареждат условно или да се зареждат при поискване чрез предложението на ECMAScript за динамичен импорт, което се поддържа в TypeScript.

Синтаксисът за динамични import изрази в TypeScript е следният:

```
async function renderWidget() {
    const container = document.getElementById('widget');
    if (container !== null) {
          const widget = await import('./widget'); // Dynamic
import
        widget.render(container);
    }
}

renderWidget();
```

“tsc –watch”

Тази команда стартира компилатора на TypeScript с параметъра `--watch`, който позволява автоматично прекомпилиране на файловете в TypeScript при всяка промяна в тях.

```
tsc --watch
```

От версия 4.9 на TypeScript, наблюдението на файлове основно разчита на събития от файловата система, автоматично преминавайки към polling, ако не може да се установи наблюдател, базиран на събития.

Non-null Assertion Operator

Non-null Assertion операторът (постфикс !), наричан още "утвърждаване за определено присвояване", е функция на TypeScript, която ви позволява да потвърдите, че дадена променлива или свойство не е null или undefined, дори ако статичният типов анализ на TypeScript предполага, че това може да е така. С тази функция е възможно да се премахнат всички изрични проверки.

```
type Person = {
    name: string;
};

const printName = (person?: Person) => {
    console.log(`Name is ${person!.name}`);
};
```

Декларации със стойности по подразбиране

Декларации със стойности по подразбиране се използват, когато на дадена променлива или параметър е присвоена стойност по подразбиране. Това означава, че ако не бъде подадена стойност за тази променлива или параметър, вместо това ще се използва стойността по подразбиране.

```
function greet(name: string = 'Anonymous'): void {
    console.log(`Hello, ${name}!`);
}
greet(); // Hello, Anonymous!
greet('John'); // Hello, John!
```

Optional Chaining

Операторът за optional chaining `?.` работи подобно на стандартния точков оператор (`.`) за достъп до свойства или методи. Разликата е, че той обработва безопасно стойности `null` или `undefined`, като прекратява израза и връща `undefined`, вместо да хвърля грешка.

```
type Person = {
    name: string;
    age?: number;
    address?: {
        street?: string;
        city?: string;
    };
};

const person: Person = {
    name: 'John',
};

console.log(person.address?.city); // undefined
```

Nullish coalescing operator

Операторът за nullish coalescing `??` връща стойността от дясната страна, ако лявата страна е `null` или `undefined`; в противен случай връща стойността от лявата страна.

```
const foo = null ?? 'foo';
console.log(foo); // foo

const baz = 1 ?? 'baz';
const baz2 = 0 ?? 'baz';
console.log(baz); // 1
console.log(baz2); // 0
```

Template Literal Types

Template Literal Types позволяват да се манипулират string стойности на ниво тип и да се генерират нови string типове на база съществуващи. Те са полезни за създаване на по-изразителни и прецизни типове при операции със string стойности.

```
type Department = 'engineering' | 'hr';
type Language = 'english' | 'spanish';
type Id = `${Department}-${Language}-id`; // "engineering-english-id" | "engineering-spanish-id" | "hr-english-id" | "hr-spanish-id"
```

Function overloading

Function overloading позволява да дефинираш множество function signatures за една и съща функция, като всяка има различни типове на параметрите и return type. Когато извикаш overloaded функция, TypeScript използва подадените аргументи, за да определи правилния function signature:

```
function makeGreeting(name: string): string;
function makeGreeting(names: string[]): string[];

function makeGreeting(person: unknown): unknown {
    if (typeof person === 'string') {
        return `Hi ${person}!`;
    } else if (Array.isArray(person)) {
        return person.map(name => `Hi, ${name}!`);
    }
    throw new Error('Unable to greet');
}

makeGreeting('Simon');
makeGreeting(['Simone', 'John']);
```

Recursive Types

Recursive Type е тип, който може да реферира сам себе си. Това е полезно за дефиниране на структури от данни с йерархична или рекурсивна структура (потенциално безкрайно вложени), като linked lists, trees и graphs.

```
type ListNode<T> = {
    data: T;
    next: ListNode<T> | undefined;
};
```

Recursive Conditional Types

В TypeScript е възможно да се дефинират сложни типови зависимости чрез логика и рекурсия. Нека го разделим на по-прости части:

Conditional Types позволяват да дефинираш типове на базата на булеви условия:

```
type CheckNumber<T> = T extends number ? 'Number' : 'Not a number';
type A = CheckNumber<123>; // 'Number'
type B = CheckNumber<'abc'>; // 'Not a number'
```

Рекурсия означава дефиниция на тип, която се реферира към самата себе си:

```
type Json = string | number | boolean | null | Json[] | { [key: string]: Json };

const data: Json = {
    prop1: true,
    prop2: 'prop2',
    prop3: {
```

```
        prop4: [],
    },
};
```

Recursive Conditional Types комбинират и двете - условна логика и рекурсия. Това означава, че дефиницията на тип може да зависи от самата себе си чрез условни проверки, което позволява много гъвкави и сложни типови отношения.

```
type Flatten<T> = T extends Array<infer U> ? Flatten<U> : T;

type NestedArray = [1, [2, [3, 4], 5], 6];
type FlattenedArray = Flatten<NestedArray>; // 2 | 3 | 4 | 5 |
1 | 6
```

ECMAScript Module Support in Node

Node.js добавя поддръжка за ECMAScript Modules от версия 15.3.0, а TypeScript поддържа ECMAScript Module Support за Node.js от версия 4.7. Това може да се активира чрез свойството `module` със стойност `nodenext` в tsconfig.json файла:

```
{
  "compilerOptions": {
    "module": "nodenext",
    "outDir": "./lib",
    "declaration": true
  }
}
```

Node.js поддържа два файлови разширения за модули: `.mjs` за ES modules и `.cjs` за CommonJS modules. Съответните TypeScript разширения са `.mts` за ES modules и `.cts` за CommonJS modules. При компилация TypeScript ще генерира `.mjs` и `.cjs` файлове.

Ако искате да използвате ES модули във вашия проект, можете да зададете стойността на свойството `type` на “module” във файла package.json. Това указва на Node.js да третира проекта като проект с ES модули.

TypeScript също поддържа type declarations чрез .d.ts файлове. Те предоставят типова информация за библиотеки или модули, което позволява type checking и auto-completion.

Assertion Functions

В TypeScript assertion functions са функции, които чрез своята връщана стойност показват дали дадено условие е изпълнено. В най-опростения си вид функцията за проверка проверява даден предикат и генерира грешка, когато предикатът се оценява като false.

```
function isNumber(value: unknown): asserts value is number {
    if (typeof value !== 'number') {
        throw new Error('Not a number');
    }
}
```

Или може да бъде деклариран като function expression:

```
type AssertIsNumber = (value: unknown) => asserts value is
number;
const isNumber: AssertIsNumber = value => {
    if (typeof value !== 'number') {
        throw new Error('Not a number');
    }
};
```

Assertion functions са подобни на type guards. Type guards се използват за runtime проверки и връщат boolean стойност, която показва дали даден тип е валиден в конкретен контекст. Разликата е, че type guards връщат false, докато assertion functions хвърлят грешка при неуспех. Функциите за проверка имат сходства с типовите защити. Типовите защити бяха въведени първоначално, за да извършват проверки по време на изпълнение и да гарантират типа на дадена стойност в рамките на конкретен обхват. По-конкретно, типовата защита е функция, която изчислява типов предикат и връща булева стойност, показваща дали предикатът е истински или неистински. Това се различава леко от функциите за проверка, при които целта е да се генерира грешка, а не да се върне стойност false, когато предикатът не е изпълнен.

Пример за типна защита:

```
const isNumber = (value: unknown): value is number => typeof
value === 'number';
```

Variadic Tuple Types

Variadic Tuple Types са feature, въведен в TypeScript версия 4.0, нека започнем да ги изучаваме, като си припомним какво е tuple:

Tuple типът е масив, който има дефинирана дължина и в който типът на всеки елемент е известен:

```
type Student = [string, number];
const [name, age]: Student = ['Simone', 20];
```

Терминът “variadic” означава неопределена арност (приема променлив брой аргументи).

Variadic tuple е tuple тип, който има всички свойства като преди, но точната форма все още не е дефинирана:

```
type Bar<T extends unknown[]> = [boolean, ...T, number];

type A = Bar<[boolean]>; // [boolean, boolean, number]
type B = Bar<['a', 'b']>; // [boolean, 'a', 'b', number]
type C = Bar<[]>; // [boolean, number]
```

В предишния код можем да видим, че формата на tuple-а се определя от generic-а `T`, който е подаден.

Variadic tuple-ите могат да приемат множество generics, което ги прави много гъвкави:

```
type Bar<T extends unknown[], G extends unknown[]> = [...T, boolean, ...G];

type A = Bar<[number], [string]>; // [number, boolean, string]
type B = Bar<['a', 'b'], [boolean]>; // ["a", "b", boolean, boolean]
```

С новите variadic tuples можем да използваме:

- Spread синтаксисът в tuple type вече може да бъде generic, което означава, че можем да моделираме higher-order операции върху tuples и arrays дори когато не знаем реалните типове, с които работим.
- Rest елементи могат да се появяват на всяка позиция в tuple.

Пример:

```ts
type Items = readonly unknown[];

function concat<T extends Items, U extends Items>(
    arr1: T,
    arr2: U
): [...T, ...U] {
    return [...arr1, ...arr2];
}

concat([1, 2, 3], ['4', '5', '6']); // [1, 2, 3, "4", "5", "6"]
```

Boxed types

Boxed types се отнасят до wrapper обекти, които се използват за представяне на primitive типове като обекти. Тези wrapper обекти предоставят допълнителна функционалност и методи, които не са налични директно върху primitive стойностите.

Когато достъпваш метод като `charAt` или `normalize` върху `string` primitive, JavaScript го обвива в `String` обект, извиква метода и след това изхвърля обекта.

Демонстрация:

```ts
const originalNormalize = String.prototype.normalize;
String.prototype.normalize = function () {
    console.log(this, typeof this);
    return originalNormalize.call(this);
};
console.log('\u0041'.normalize());
```

TypeScript представя тази разлика чрез отделни типове за primitive стойностите и техните object wrapper-и:

- string => String

- number => Number
- boolean => Boolean
- symbol => Symbol
- bigint => BigInt

Boxed types обикновено не са необходими. Избягвай използването им и вместо това използвай primitive типове — например `string` вместо `String`.

Ковариантност и Контравариантност в TypeScript

Ковариантността и контравариантността описват отношенията между типовете в генеричните типове. `Dog` е `Animal`, но не всеки `Animal` е `Dog`.

Ковариантност (връщане на стойности): функция, която връща `Dog`, може да замени функция, връщаща `Animal`. Всяко куче е животно, но функция за произволно животно може да върне котка.

```
class Animal {
    name = '';
}

class Dog extends Animal {
    breed = '';
}

type Producer<T> = () => T;

declare let produceAnimal: Producer<Animal>;
declare let produceDog: Producer<Dog>;

produceAnimal = produceDog; // Valid
produceDog = produceAnimal; // Error
```

Контравариантност (приемане на стойности): функция, която приема всеки `Animal`, може да замени функция, приемаща `Dog`. Тя обработва и кучета; функция само за кучета не може безопасно да обработва всички животни.

```
type Consumer<T> = (value: T) => void;

declare let consumeAnimal: Consumer<Animal>;
declare let consumeDog: Consumer<Dog>;

consumeDog = consumeAnimal; // Valid
consumeAnimal = consumeDog; // Error with strictFunctionTypes
```

Стрелките показват посоката на безопасното присвояване:

- Dog -> Animal
- Producer<Dog> -> Producer<Animal> (covariance)
- Consumer<Animal> -> Consumer<Dog> (contravariance)

Променяеми масиви: TypeScript позволява също `Dog[]` да се присвои на `Animal[]`. Двете променливи сочат един и същ масив; през по-общия тип може да се добави `Animal`, което не е куче.

```
const dogs: Dog[] = [new Dog()];
const animals: Animal[] = dogs;

animals.push(new Animal()); // Allowed, but unsafe
// dogs now contains an Animal that is not a Dog.
```

Настройка на компилатора: с `strictFunctionTypes` параметрите на самостоятелните функции се проверяват контравариантно. Без тази настройка TypeScript може да позволи и двете посоки (бивариантност), макар да е

опасно. Параметрите в декларации на методи и конструктори са изключение.

Optional Variance Annotations for Type Parameters

От TypeScript 4.7 `out` означава ковариантност, `in` — контравариантност, а `in out` — инвариантност. TypeScript обикновено извежда вариантността. Анотациите трябва да съответстват на употребата на генеричния параметър, а не произволно да я променят.

```
type Producer<out T> = () => T;
type Consumer<in T> = (value: T) => void;
type Transformer<in out T> = (value: T) => T;
```

Template String Pattern Index Signatures

Template string pattern index signatures ни позволяват да дефинираме гъвкави index signatures чрез template string patterns. Тази функционалност ни позволява да създаваме обекти, които могат да бъдат индексирани със специфични шаблони от string keys, като по този начин получаваме повече контрол и прецизност при достъп и манипулация на свойства.

TypeScript от версия 4.4 позволява index signatures за symbols и template string patterns.

```
const uniqueSymbol = Symbol('description');

type MyKeys = `key-${string}`;

type MyObject = {
    [uniqueSymbol]: string;
```

```
    [key: MyKeys]: number;
};

const obj: MyObject = {
    [uniqueSymbol]: 'Unique symbol key',
    'key-a': 123,
    'key-b': 456,
};

console.log(obj[uniqueSymbol]); // Unique symbol key
console.log(obj['key-a']); // 123
console.log(obj['key-b']); // 456
```

The satisfies Operator

`satisfies` операторът позволява да провериш дали даден тип отговаря на специфичен interface или условие. С други думи, той гарантира, че типът има всички изисквани свойства и методи на даден interface. Това е начин да се гарантира, че дадена променлива съответства на дефиниция на тип. Ето един пример:

```
type Columns = 'name' | 'nickName' | 'attributes';

type User = Record<Columns, string | string[] | undefined>;

// Type Annotation using `User`
const user: User = {
    name: 'Simone',
    nickName: undefined,
    attributes: ['dev', 'admin'],
};

// В следните редове TypeScript няма да може да направи правилно inference
```

```
user.attributes?.map(console.log); // Property 'map' does not exist on type 'string | string[]'. Property 'map' does not exist on type 'string'.
user.nickName; // string | string[] | undefined

// Type assertion с `as`
const user2 = {
    name: 'Simon',
    nickName: undefined,
    attributes: ['dev', 'admin'],
} as User;

// И тук TypeScript няма да може да направи правилно inference
user2.attributes?.map(console.log); // Property 'map' does not exist on type 'string | string[]'. Property 'map' does not exist on type 'string'.
user2.nickName; // string | string[] | undefined

// Използвайки `satisfies` операторите, можем да направим правилно inference на типовете
const user3 = {
    name: 'Simon',
    nickName: undefined,
    attributes: ['dev', 'admin'],
} satisfies User;

user3.attributes?.map(console.log); // TypeScript инферира правилно: string[]
user3.nickName; // TypeScript инферира правилно: undefined
```

Type-Only Imports и Export

Type-Only Imports и Export позволява да импортираш или експортираш типове без да импортираш или експортираш стойностите или функциите, свързани с тези типове. Това може да бъде полезно за намаляване на размера на bundle-а.

За type-only imports можеш да използваш ключовата дума `import type`.

TypeScript позволява използване както на declaration, така и на implementation файлови разширения (.ts, .mts, .cts и .tsx) в type-only imports, независимо от настройката `allowImportingTsExtensions`.

Например:

```
import type { House } from './house.ts';
```

Поддържат се следните форми:

```
import type T from './mod';
import type { A, B } from './mod';
import type * as Types from './mod';
export type { T };
export type { T } from './mod';
```

using declaration и Explicit Resource Management

`using` декларация е block-scoped, immutable binding, подобен на `const`, използван за управление на disposable ресурси. Когато бъде инициализиран със стойност, методът `Symbol.dispose` на тази стойност се записва и впоследствие се изпълнява при излизане от съответния block scope.

Това е базирано на ECMAScript Resource Management функционалност, която е полезна за извършване на важни cleanup операции след създаване на обекти, като затваряне на връзки, изтриване на файлове и освобождаване на памет.

Бележки:

- Поради скорошното му въвеждане в TypeScript версия 5.2, повечето runtime среди нямат нативна поддръжка. Ще са ви необходими полифили за: `Symbol.dispose`, `Symbol.asyncDispose`, `DisposableStack`, `AsyncDisposableStack`, `SuppressedError`.
- Освен това, ще трябва да конфигурирате вашия tsconfig.json както следва:

```
{
    "compilerOptions": {
        "target": "es2022",
        "lib": ["es2022", "esnext.disposable", "dom"]
    }
}
```

Пример:

```
//@ts-ignore
Symbol.dispose ??= Symbol('Symbol.dispose'); // Simple polyfill

const doWork = (): Disposable => {
    return {
        [Symbol.dispose]: () => {
            console.log('disposed');
        },
    };
};

console.log(1);

{
    using work = doWork(); // Resource is declared
    console.log(2);
} // Resource is disposed (e.g., `work[Symbol.dispose]()` is
evaluated)

console.log(3);
```

Кодът ще изведе:

```
1
2
disposed
3
```

Resource, който е eligible за disposal, трябва да отговаря на `Disposable` interface:

```
// lib.esnext.disposable.d.ts
interface Disposable {
    [Symbol.dispose](): void;
}
```

`using` декларациите записват disposal операциите в stack, като гарантират, че те се изпълняват в обратен ред на декларацията:

```
{
    using j = getA(),
        y = getB();
    using k = getC();
} // disposes `C`, then `B`, then `A`.
```

Resources се гарантира, че ще бъдат disposed, дори ако последващ код или exception възникне. Това може да доведе до ситуация, в която disposal хвърля exception, който може да suppress-не друг. За да се запази информация за suppressed errors, се въвежда нов native exception `SuppressedError`.

Декларация await using

Декларацията `await using` се използва за управление на асинхронно освобождаеми ресурси. Стойността трябва да разполага с метод `Symbol.asyncDispose`, който ще бъде извикан в края на блока.

```
async function doWorkAsync() {
    await using work = doWorkAsync(); // Resource is declared
}     //     Resource     is     disposed     (e.g.,     `await
work[Symbol.asyncDispose]()` is evaluated)
```

За да бъде ресурсът асинхронно освобождаем, той трябва да отговаря на интерфейса `Disposable` или `AsyncDisposable`:

```
// lib.esnext.disposable.d.ts
interface AsyncDisposable {
    [Symbol.asyncDispose](): Promise<void>;
}

//@ts-ignore
Symbol.asyncDispose   ??=   Symbol('Symbol.asyncDispose');   //
Simple polyfill

class DatabaseConnection implements AsyncDisposable {
     // Метод, който се извиква, когато обектът се унищожава
асинхронно
    [Symbol.asyncDispose]() {
        // Затваряне на връзката и връщане на promise
        return this.close();
    }

    async close() {
        console.log('Closing the connection...');
             await new Promise(resolve => setTimeout(resolve,
1000));
        console.log('Connection closed.');
    }
}
```

```
async function doWork() {
      // Създаване на нова връзка и асинхронно освобождаване, когато излезе от обхвата
       await using connection = new DatabaseConnection(); // Resource е деклариран
    console.log('Doing some work...');
}     //     Resource     е     освободен     (например,     `await connection[Symbol.asyncDispose]()` се извиква)

doWork();
```

Кодът ще изведе:

```
Doing some work...
Closing the connection...
Connection closed.
```

Декларациите `using` и `await using` са разрешени в изрази: `for`, `for-in`, `for-of`, `for-await-of`, `switch`.

Import Attributes

Import Attributes в TypeScript 5.3 (labels за imports) казват на runtime-а как да обработи модули (JSON и др.). Това подобрява сигурността, като гарантира ясни imports и се съобразява с Content Security Policy (CSP) за по-безопасно зареждане на ресурси. TypeScript валидира тяхната коректност, но оставя интерпретацията на runtime-а.

Пример:

```
import config from './config.json' with { type: 'json' };
```

с dynamic import:

```
const config = import('./config.json', { with: { type: 'json' } });
```

Проверка на синтаксиса на регулярните изрази

От версия 5.5.4 на TypeScript той проверява литералите с регулярни изрази за често срещани грешки още по време на компилация (например невалиден синтаксис, грешни обратни препратки, неподдържани функции за целевата версия на JavaScript). Това помага за ранното откриване на грешки, но не проверява низовете от типа new RegExp(“...”).

```
let r = /(a)\2/; // Error: This backreference refers to a group that does not exist.
```

import defer

`import defer` ви позволява да заредите модул, но да забавите неговото изпълнение, докато всъщност не използвате нещо от него. Това помага да се избегне ненужна работа и странични ефекти.

- Работи само с: `import defer * as name from "module"`
- Кодът се изпълнява само когато достъпите експорта